

ProphetBoost: A hybrid pipeline for accurate and transparent electricity load forecasting

Nazrul Amin ^a, Yong-Woon Kim ^a, Yung-Cheol Byun ^b.*

^a Department of Computer Engineering, Jeju National University, Jeju 63243, South Korea

^b Department of Computer Engineering, Major of Electronic Engineering, Food Tech Center (FTC), Jeju National University, Jeju 63243, South Korea

ARTICLE INFO

Keywords:

Short-term load forecasting
Hybrid model
Feature selection
Explainable Artificial Intelligence (XAI)
Shap analysis
Smart grids and urban energy systems

ABSTRACT

Accurate short-term load forecasting is critical for the efficient operation of building energy systems and smart grids. However, increasing demand variability from weather fluctuations, occupant behavior, and renewable energy integration challenges conventional statistical approaches. Meanwhile, advanced machine learning models often achieve high accuracy but lack interpretability, limiting their adoption in building energy management. This study proposes ProphetBoost, a novel hybrid forecasting pipeline that integrates Prophet's additive decomposition with an XGBoost regressor, enhanced by an explainable AI (XAI)-driven embedded-stability feature selection strategy. Using eight years of hourly electricity consumption data from Jeju Island and meteorological observations from four weather stations, the pipeline generates engineered features and selects a compact, stable subset of 14 predictors from 111 candidates. Experimental results show that ProphetBoost achieves a 16.77% improvement in test Mean Absolute Error (from 6.44 MW to 5.36 MW) compared with baseline models. SHapley Additive exPlanation (SHAP) based interpretability confirms that recent load dynamics and weather covariates are key drivers of building and regional demand. By combining predictive accuracy with transparency, ProphetBoost provides an effective tool for building energy management, demand response, and operational decision support in smart urban environments.

1. Introduction

Accurate short-term load forecasting is essential for the effective management and reliability of power systems, directly influencing critical operations such as unit commitment, reserve scheduling, energy trading, and consumption planning. The increasing complexity of demand patterns driven by economic expansion, fluctuating weather conditions, evolving consumer behaviors, and the growing integration of solar panels and electric vehicles introduces nonlinearities and abrupt shifts that challenge traditional statistical models [1]. Even marginal improvements in forecast accuracy can yield substantial cost savings for utilities, making the continuous advancement of forecasting methodologies a high-priority research direction [2]. In recent years, artificial intelligence (AI) and machine learning (ML) approaches, such as gradient boosting and deep neural networks, have outperformed classical methods in predictive accuracy [3,4]. However, the increasing complexity of these models often compromises their transparency and interpretability. While simpler models, such as linear regression or shallow decision trees, provide human-readable explanations, deep architectures with millions of parameters typically operate as opaque black boxes [5]. This lack of transparency poses significant challenges for model validation, regulatory compliance, and operator trust, particularly in high-stakes energy

* Corresponding author.

E-mail addresses: nazrulamin@stu.jejunu.ac.kr (N. Amin), ywkim@jejunu.ac.kr (Y.-W. Kim), ycb@jejunu.ac.kr (Y.-C. Byun).

<https://doi.org/10.1016/j.compeleceng.2026.111280>

Received 22 October 2025; Received in revised form 16 April 2026; Accepted 1 June 2026

0045-7906/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

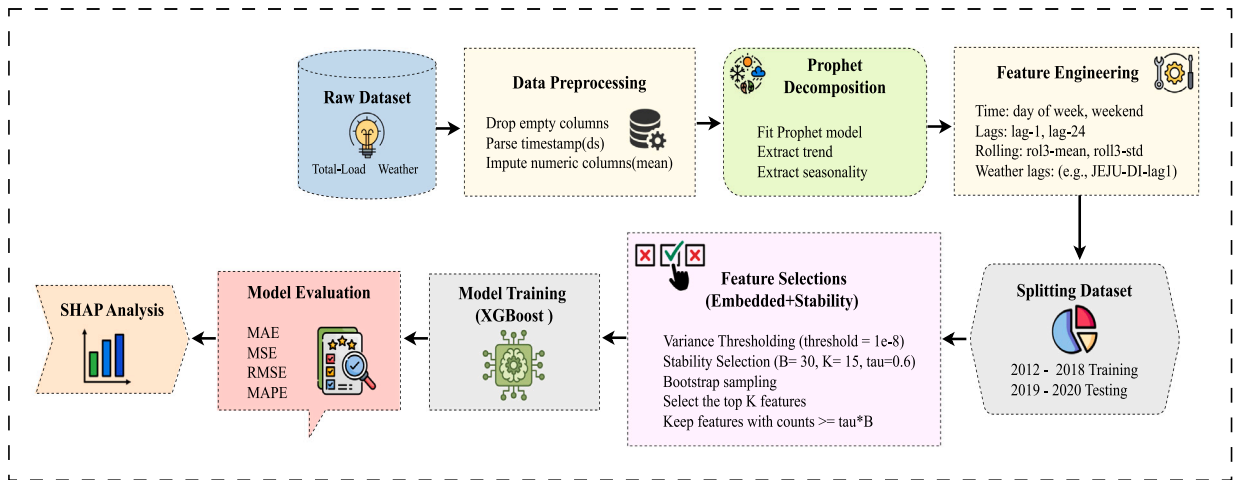


Fig. 1. Comprehensive load-forecasting workflow of the proposed ProphetBoost pipeline.

applications, where understanding the reasoning behind predictions is as important as the predictions themselves. Explainable Artificial Intelligence (XAI) techniques address this issue by linking model predictions to input features in human-interpretable ways, thereby enhancing transparency and trust [6,7]. Beyond providing post-hoc explanations, XAI-driven feature attribution can guide feature selection by identifying and retaining only the most relevant predictors. This not only improves computational efficiency and model generalization but also enhances interpretability while maintaining strong predictive performance [8]. Feature selection methods such as embedded techniques (e.g., feature importance from tree-based models) and stability selection (e.g., bootstrap-based ranking) are particularly valuable for multivariate time-series forecasting, as they filter out irrelevant or redundant inputs, reducing model complexity and improving transparency [9,10]. However, the integration of XAI-based feature selection into energy load forecasting remains underexplored.

This study introduces a novel hybrid forecasting approach, termed *ProphetBoost*, that integrates Prophet's additive decomposition with an XGBoost regressor, augmented by an embedded-stability feature-selection scheme. Unlike conventional hybrid models that loosely combine statistical and machine learning techniques, the proposed ProphetBoost framework introduces a structured decomposition-residual learning paradigm designed for both interpretability and robustness. Prophet is first employed to explicitly model trend, seasonality, and holiday effects, thereby isolating the structured components of electricity demand. The residual component is then modeled using XGBoost to capture nonlinear interactions between meteorological variables and engineered temporal features. Furthermore, we integrate an embedded-stability feature selection mechanism within the residual learning stage to enhance robustness and reduce overfitting. This principled integration ensures a leakage-free, interpretable, and stability-validated forecasting pipeline, moving beyond simple model stacking toward a transparent and operationally reliable forecasting framework. The overall process for short-term load forecasting is illustrated in Fig. 1, beginning with raw data ingestion and Prophet-based decomposition, followed by feature engineering, two-stage feature selection, XGBoost-based model training, and final performance evaluation. Using eight years of hourly load data from Jeju Island combined with meteorological data from four local weather stations, the contributions of this study are summarized as follows:

- **Hybrid Forecasting Pipeline:** Integration of Prophet's capability to decompose time series into trend and seasonal patterns with XGBoost's powerful regression, thereby capturing both structural and nonlinear relationships in load data.
- **Two-Stage XAI-Based Feature Selection:** Combination of variance thresholding with embedded stability selection using SHAP and XGBoost across bootstraps to identify a robust, interpretable feature subset.
- **Enhanced Accuracy and Interpretability:** The proposed pipeline achieves a 16.77% improvement in test Mean Absolute Error (MAE) (from 6.44 MW to 5.36 MW) compared to a baseline model, with SHAP-based analyses confirming that the selected features capture critical load dynamics and meteorological influences.
- **Practical Applicability:** By balancing predictive accuracy with interpretability, the framework supports model validation, ensures regulatory compliance, and fosters operational trust, making it effective for real-world energy forecasting.

Unlike conventional approaches that rely solely on decomposition or ML models, the proposed pipeline leverages Prophet to extract interpretable trend and seasonal components, which are then enriched with engineered features (calendar indicators, lagged loads, rolling statistics, and weather covariates). The embedded-stability feature-selection process ensures that only the most impactful and stable predictors are retained, addressing the trade-off between model complexity and interpretability. This synergy produces a forecasting model that is both accurate and transparent, as validated by SHAP-based insights, marking a significant advancement in the domain of short-term load forecasting.

2. Related work

2.1. Traditional feature selection methods

Feature selection is the process of identifying and retaining the most informative and relevant predictors from a dataset in order to enhance model accuracy, reduce overfitting, and improve interpretability [11]. Broadly, feature selection approaches can be categorized into three groups [12]. Filter methods evaluate the relevance of each feature independently of the learning algorithm, often relying on statistical metrics such as Pearson correlation, mutual information, or distance-based measures. These methods are computationally efficient and well-suited for high-dimensional datasets, though they may fail to capture interactions between features that are critical for specific models [13]. Wrapper methods frame feature selection as a search problem in which subsets of features are iteratively assessed by training and validating the model (e.g., forward or backward selection). While wrappers often yield superior predictive performance by accounting for feature interactions, they are computationally demanding, especially when dealing with large feature spaces [14]. Embedded methods integrate feature selection directly into the model training process, such as decision trees, random forests, gradient boosting, and regularized linear models like LASSO, which shrink some coefficients to zero [15]. Finally, hybrid approaches combine multiple strategies (e.g., a fast filter followed by an embedded or wrapper method) to exploit complementary strengths and balance efficiency with selection quality.

2.2. Feature selection in load forecasting

In [16], a hybrid feature selection (RMR-HFS) combines two filter methods (RReliefF and mutual information) with a wrapper method (recursive feature elimination) to identify the eight most relevant features for an LSTM model, reducing dimensionality and overfitting and yielding lower MAPE and RMSE than MLP and RNN baselines. In [17], the non-linear RReliefF algorithm is used to remove redundant exogenous features (weather, calendar, price, etc.), enabling a multivariate load-forecasting system (MCLFS) that combines ANNs and DNNs with an Adaptive Benchmark Model Selection (ABMS) scheme to select the best sub-predictors. Jiang et al. [18] propose a Holistic Feature Selection Method (HFSM) that leverages a feedforward LSTM to compute a feature-importance matrix capturing relevance, redundancy, and interactions, then applies a two-sample t-test and a wrapper-style search, achieving an average 12.1% improvement in short-term load-forecast accuracy across various ANN models. In [19], a fast correlation-based filter (threshold 0.8) eliminates redundant statistical and frequency-domain features for real-time load identification in a solar-powered aircraft EDS, leading to 92.68% accuracy with a hyperparameter-tuned XGBoost classifier, though validation is limited to a controlled HIL setup. In [20], a Multi-Approach Feature Selection (MAFS) scheme combines Pearson correlation, random forest feature importance, and domain-expert judgment to prune redundant variables, reducing dimensionality and overfitting while improving PGBM training speed and generalization.

2.3. XAI-based feature selection

XAI-driven feature selection approaches move beyond traditional filter, wrapper, and embedded paradigms by reframing feature importance as an interpretability problem. Instead of retraining models, these methods directly extract contribution scores from trained models [21]. Such techniques can be categorized into model-specific methods, which leverage a model's internal mechanisms (e.g., decision-path analysis in tree-based models or attention weights in neural networks), and model-agnostic methods, which treat the trained model as a black box and infer feature contributions through input-output analysis [22]. Among model-agnostic approaches, SHAP (SHapley Additive exPlanations) is widely recognized for assigning each feature a contribution value derived from Shapley values in cooperative game theory [23]. SHAP offers local accuracy, consistency, and missingness, ensuring that contributions sum to the model output, increased influence cannot reduce a feature's importance, and absent features receive zero contribution [24]. Practical implementations include KernelSHAP, which approximates Shapley values via weighted linear regression on perturbed samples, and TreeSHAP, which exploits tree-ensemble structure to compute exact Shapley values in polynomial time, making SHAP both theoretically grounded and computationally tractable across diverse model classes [25].

2.4. Energy forecasting models

Energy forecasting has evolved from traditional statistical models such as ARIMA and SARIMAX, which are interpretable but limited in capturing nonlinear dynamics and long-range dependencies [31–33], to machine-learning approaches including support vector regression, Random Forests, and gradient-boosted trees that improve predictive accuracy but often rely on manual feature engineering [34,35]. Deep-learning architectures such as LSTM and GRU further enhanced performance by modeling temporal dependencies directly [36], while Transformer-based frameworks (e.g., Temporal Fusion Transformer and Autoformer) employ attention mechanisms to capture complex patterns, albeit with increased computational and tuning demands [37,38]. Hybrid approaches including ARIMA–ANN combinations, PCA-enhanced XGBoost ensembles, and CNN–LSTM stacks have also been explored to combine complementary strengths across modeling paradigms. Among recent advances, the Temporal Fusion Transformer (TFT) [37] enables multi-horizon forecasting by integrating static, known future, and historical inputs within a unified attention framework. Its Variable Selection Network provides intrinsic interpretability through dynamic feature weighting. Similarly, Informer [39] improves long-sequence forecasting efficiency using probabilistic sparse self-attention mechanisms. Recent

Table 1

Comprehensive comparison of recent studies on electricity, and multi-energy forecasting. The table summarizes dataset characteristics, modeling techniques, principal outcomes, and reported limitations, with each study indexed by reference number for concise citation.

Ref	Year	Dataset/Case	Models used	Key findings	Main limitations
[26]	2025	Tehran smart-grid hourly load for 81 substations (2015–2019)	SAGAN-ConvLSTM; GRU, LSTM, ConvLSTM, CTS-LSTM, SA-ConvLSTM	Semivariogram-guided GAN imputes missing data; ConvLSTM gives best 20-day STLTF (lowest MAE/RMSE vs deep baselines).	Single city; needs coordinates and dense data; point forecasts only, no probabilistic or cross-grid tests.
[20]	2024	Delhi feeder 5-min load (Feb 2021–2022) with weather + calendar features	PGBM + MAFS + ISTD; ARIMA, SVR, RBFNN, LSTM, BiLSTM, CNN-LSTM, BNN	Probabilistic STLTF with calibrated prediction intervals and very high accuracy ($R^2 \approx 0.997$) and relatively fast training.	Single feeder and one year; seasonal periods chosen manually; no Transformer or linear SOTA baselines.
[18]	2023	Swiss daily load + daily weather (2015–2017)	F-LSTM based HFSM + LSTM/FC/CNN models	HFSM reduces features (32 → 11) and improves ANN forecast error by about 12.1% on the test period.	Only one daily dataset; no tree/boosting models; short evaluation window.
[16]	2022	Switzerland hourly load + 21 weather variables (2008–2012)	RReliefF + MI + RFE + LSTM	Hybrid RMR-HFS selects 8 key variables and LSTM on this subset yields the best RMSE/MAPE under rolling validation.	ANN-only baselines; deterministic forecasts; no boosting or Transformer models.
[1]	2024	Brazilian ONS + ISO-NE very short-term (sub-hourly, 2015–2019)	STL/EMD/CEEMDAN/EWT + GBR/XGBoost/kNN/SVR + ARD stacking	Decomposition plus AutoML-tuned ensembles significantly improve VSTLTF accuracy (e.g., RMSE = 1931.8 MW on ONS).	Focus on VSTLTF (minutes-ahead), deterministic only; no deep-learning baselines; high offline computation.
[27]	2024	Hourly PV output + weather (440 days, China)	TIME + TFT; compared with DeepAR and TFT variants	TIME-TFT achieves the best MAE/RMSE and well-calibrated prediction intervals with interpretable feature and time importance.	Single PV site; limited probabilistic baselines; no cross-climate or multi-site validation.
[28]	2025	Short-term load forecasting with weather + calendar features (hourly regional datasets)	SALF (Self-Adaptive Learning Framework); compared with LSTM, GRU, XGBoost	Adaptive feature weighting and dynamic parameter tuning improve MAE/RMSE and robustness across seasonal variations.	Deterministic forecasts only; higher computational cost due to adaptive updates; no decomposition-residual hybrid modeling.
[29]	2024	Hourly load forecasting with weather + calendar features	Transformer-based hybrid deep learning model	Improves MAE/RMSE by capturing long-range temporal dependencies.	High computational cost; deterministic only; limited interpretability and cross-region validation.
[30]	2025	US sector-wise monthly peak demand (50 years) + NSW daily load	EMD + TL ResNet-18; stacked ARIMA, Prophet, GRU, TFT	Hybrid EMD + stacking attains $\approx 1\%$ MAPE and outperforms N-BEATS, DLinear, edRVFL and other SOTA baselines.	No spatial modeling or uncertainty; complex, compute-intensive pipeline.

studies have also explored Prophet-based hybrid models with recurrent neural networks, such as Prophet-LSTM [40] and Prophet-BiLSTM [41], for load and microgrid energy forecasting. These works show that Prophet-assisted decomposition can effectively complement deep sequence models in capturing nonlinear temporal dynamics. Prophet combined with recurrent deep learning models, such as LSTM and BiLSTM, can improve forecasting accuracy and support more effective energy management in microgrid systems. As summarized in Table 1, recent forecasting studies increasingly employ deep hybrid architectures, decomposition-based preprocessing, and advanced feature-selection or XAI-based filtering to enhance accuracy. However, most prior works remain limited to single-region datasets, emphasize deterministic forecasts, and rarely evaluate tree-based boosting models in combination with principled temporal decomposition. Despite recent progress in short-term load forecasting, an important methodological gap remains. Existing studies have shown the value of combining temporal decomposition with machine learning or deep learning to improve predictive accuracy; however, many of these approaches rely on computationally intensive deep architectures, which may reduce transparency and limit operational deployment. In addition, decomposition is often used only as a preprocessing step, rather than being integrated into a structured decomposition-residual learning framework with a rigorous mechanism to retain only stable and interpretable predictors. As a result, the literature still lacks a forecasting pipeline that simultaneously couples interpretable trend-seasonality decomposition, nonlinear residual modeling, and stability-guided feature selection within a leakage-free and operationally efficient design. The proposed ProphetBoost framework is intended to fill this gap by combining Prophet-based

Table 2

Variable description and corresponding units for demand and weather-related predictors.

Category	Variable name	Description	Unit/Range
Demand	TOTAL_LOAD	Electricity load demand	MW
Weather Features	TA	Ambient temperature	°C
	TD	Dew point temperature	°C
	HM	Relative humidity	%
	WS	Wind speed	m/s
	WD_DEG	Wind direction	Degrees (0–360)
	PA	Atmospheric pressure	hPa
	DI	Dew point index	°C
	ST	Snow thickness or similar metric	cm (tentative)
	SI	Solar irradiance or similar metric	W/m ² (tentative)

Table 3

Descriptive statistics of the Jeju Island hourly electricity load dataset, summarizing central tendency, variability, and demand range.

Metric	Value
Mean Load	578.63 MW
Standard Deviation	108.72 MW
Minimum	0 MW
Maximum	1018 MW
Coefficient of Variation (CV)	0.188 (18.8%)

Table 4

Top positive and negative correlations between meteorological variables and electricity load.

Top positive		Top negative	
Feature	Corr.	Feature	Corr.
GOSAN_PA	+0.175	SUNGSAN_HM	−0.168
JEJU_PA	+0.173	GOSAN_ST	−0.167
SUNGSAN_PA	+0.171	SUNGSAN_ST	−0.161
SEOGWIPO_PA	+0.163	SEOGWIPO_ST	−0.155
GOSAN_WS	+0.135	SEOGWIPO_TA	−0.148

decomposition, XGBoost residual learning, and embedded–stability feature selection to achieve predictive accuracy, transparency, and practical robustness in short-term electricity load forecasting.

3. Dataset overview

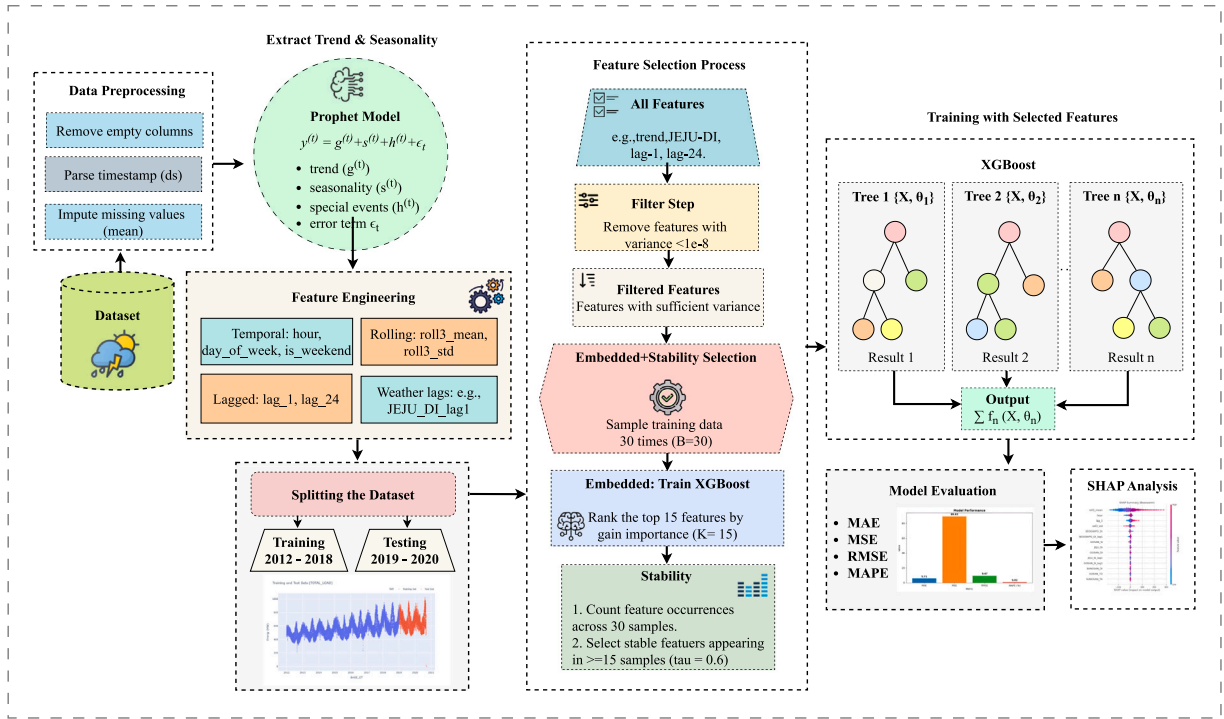
In this study, we employ hourly electricity-load data for Jeju Island, South Korea, collected between 2012 and 2020, alongside meteorological measurements from four local weather stations Jeju-si, Gosan, Seongsan, and Seogwipo [42]. The load series (TOTAL_LOAD) represents the aggregate regional demand in megawatts (MW). Each station provides nine weather covariates: ambient temperature (TA, °C), dew-point temperature (TD, °C), relative humidity (HM, %), wind speed (WS, m/s), wind direction (WD_DEG, °), atmospheric pressure (PA, hPa), dew-point index (DI, °C), snow thickness or analogous metric (ST, cm), and solar irradiance or analogous metric (SI, W/m²) which serve as predictors in our forecasting model. As detailed in Table 2, these variables are categorized into *Demand* and *Weather Features*, with their respective units and descriptions.

Table 3 summarizes the statistical characteristics of the load series. The mean demand is 578.63 MW with a standard deviation of 108.72 MW, resulting in a coefficient of variation (CV) of 18.8%, indicating moderate load volatility. The load ranges from 0 MW to 1018 MW, confirming substantial variability and the presence of high-demand peak events within the dataset. Table 4 presents the strongest positive and negative linear correlations between meteorological variables and electricity load. Atmospheric pressure variables (PA) exhibit mild positive correlations, whereas humidity (HM), surface temperature (ST), and air temperature (TA) generally show negative correlations with load. The moderate magnitude of the correlations ($|r| \leq 0.18$) suggests that the relationships between weather conditions and electricity demand are likely nonlinear, thereby supporting the use of nonlinear machine learning models such as XGBoost for forecasting. Each station logs nine variables: ambient temperature (TA), dew-point temperature (TD), relative humidity (HM), wind speed (WS), wind direction (WD_DEG), atmospheric pressure (PA), dew-point index (DI), snow thickness or analogous solid-precipitation metric (ST), and solar irradiance or analogous insolation metric (SI). Together, these 36 weather covariates and the single load variable form a 37-dimensional dataset.

Table 5

Dataset partitioning strategy for chronological evaluation without temporal leakage.

Split	Period	Years	Samples
Training	2012–2018	7	52,560
Test	2019–2020	2	17,520
Total	2012–2020	9	70,080

**Fig. 2.** Illustrates the full pipeline of the proposed forecasting system, from raw data preprocessing and Prophet-based decomposition to feature selection, XGBoost training, and SHAP-based interpretability.

3.1. Dataset partitioning and benchmark configuration

To ensure a transparent and leakage-free evaluation protocol, all experiments in this study adopt a strictly chronological data-partitioning strategy. The full benchmark dataset comprises eight years (2012–2020) of hourly electricity demand for Jeju Island, combined with 36 meteorological covariates from four weather stations (Jeju-si, Gosan, Seongsan, and Seogwipo). The preprocessed and feature-engineered dataset is divided chronologically as shown in Table 5. The training set (2012–2018, $n_{\text{train}} = 52,560$ samples) is used for model fitting, feature selection, and internal validation, while the test set (2019–2020, $n_{\text{test}} = 17,520$ samples) is held out exclusively for final evaluation. Validation procedures are model-specific: deep-learning and linear baselines (Transformer, DLinear, CNN-LSTM, CNN-ANN, Deep RVFL) are validated via Optuna hyperparameter tuning optimization within the training window with 20 epochs and validation MSE, while ProphetBoost uses 5-fold time-series cross-validation with `xgb.cv`. No random shuffling is applied at any stage to prevent temporal leakage, and all benchmark models are trained on identical engineered features and temporal splits, ensuring fair and reproducible comparative evaluation.

4. Methodology

The complete time series forecasting pipeline, which combines data preprocessing, feature engineering, feature selection, model training, and evaluation using the XGBoost method, is depicted in Fig. 2. The procedure starts with a dataset that has undergone data preprocessing, which includes removing empty columns, parsing timestamps, and using the mean to impute missing values.

Trend ($g(t)$), yearly seasonality ($s(y)$), weekly seasonality ($s(w)$), and special events ($e(t)$) are among the components that are produced when the preprocessed data is fed into a Prophet model to extract trends and seasonality. Next, the feature engineering phase incorporates temporal features (e.g., day of week, hour, weekend), rolling statistics (e.g., mean, standard deviation), and lagged variables (e.g., lag 1, lag 24), along with weather data. To prevent information leakage, a strict chronological training

protocol was followed. The dataset was first divided into training (2012–2018), validation (2019), and test (2020) sets prior to any decomposition or feature engineering. Prophet was fitted exclusively on the training data to estimate trend and seasonal components. The learned parameters were then used to generate corresponding components for validation and test periods without refitting. All lag-based and derived features were constructed using only historical observations available at each timestamp. The test set remained completely unseen until the final evaluation. The feature selection process involves an embedded stability selection method, utilizing XGBoost to rank the top 15 features by importance. A filter step removes features with variance $< 10^{-6}$. The pipeline culminates in training an XGBoost model with the selected features, producing multiple decision trees (e.g., Tree 1: X_k, β_k ; Tree 2: X_{β_j} ; Tree n : X_{β_j}) that contribute to the final prediction ($\sum f_k(x_{\beta_j})$). The model evaluation phase assesses performance using metrics such as Mean Absolute Error (MAE), Root Mean Square Error (RMSE), and Mean Absolute Percentage Error (MAPE), displayed in a bar chart for comparison. The entire process is framed to emphasize the iterative selection and embedding steps, ensuring robust feature sets for accurate forecasting.

4.1. Data preparation and feature engineering

The data preparation phase begins with the integration of hourly electricity demand and meteorological observations collected from Jeju Island, South Korea, spanning the period 2012–2020. The raw dataset includes a timestamp column (BASE_DT), which is parsed into a standardized datetime format $d_s \in \mathbb{R}^T$, where T denotes the total number of hourly observations. Columns containing entirely missing values are removed during the initial screening stage. The overall missing rate across the dataset was 7.74%, primarily driven by a small number of variables exhibiting structural missingness (e.g., SI-related features). Excluding these variables, most meteorological features had missing rates below 1%, while the target variable $y = \text{TOTAL_LOAD}$ exhibited only 0.28% missing values, indicating high data completeness. The remaining numerical variables, including the target, are imputed using univariate mean imputation [43].

$$x_i^{\text{imputed}} = \begin{cases} x_i, & \text{if } x_i \text{ is not missing} \\ \frac{1}{N} \sum_{j=1}^N x_j, & \text{otherwise} \end{cases} \quad (1)$$

This ensures consistency in downstream modeling and preserves statistical integrity across input variables. Outliers were identified using the interquartile range (IQR) method. For each feature X_j , the lower and upper bounds were defined as

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR}, \quad (2)$$

$$\text{Upper Bound} = Q_3 + 1.5 \times \text{IQR}, \quad (3)$$

where Q_1 and Q_3 denote the first and third quartiles, respectively, and

$$\text{IQR} = Q_3 - Q_1. \quad (4)$$

Importantly, extreme load values were retained, as they correspond to genuine peak-demand events critical for forecasting robustness. Only implausible meteorological anomalies, if detected, were corrected. This strategy preserves operational extremes while mitigating measurement artifacts. To extract latent time-dependent patterns from the target series, we employ the Prophet [44] decomposition model:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t \quad (5)$$

where $g(t)$ captures non-periodic long-term trends, $s(t)$ represents multiple seasonal components (daily, weekly, yearly), $h(t)$ accounts for holiday effects or special events, and $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$ is the i.i.d. residual error term. From this decomposition, the estimated trend $\hat{g}(t)$ and seasonality $\hat{s}(t)$ components are retained as engineered features to enhance model expressivity. We further construct high-informative features to encode short-term dynamics and temporal dependencies in the dataset. Temporal encodings include the hour of the day $h_t = \text{ds.hour}$, the day of the week $d_t = \text{ds.dayofweek} \in \{0, \dots, 6\}$, and a binary weekend indicator $w_t = \mathbb{1}\{d_t \in \{5, 6\}\}$. To capture autocorrelations in the load series, we introduce lag features such as the 1-hour lag ($\text{lag}_1 = y_{t-1}$) and the 24-hour lag ($\text{lag}_{24} = y_{t-24}$). Additionally, we calculate rolling statistics over a 3-hour window to capture recent load behavior [45]. These include the rolling mean, defined as $\text{roll3_mean}_t = \frac{1}{3} \sum_{i=t-2}^t y_i$, and the rolling standard deviation, computed as $\text{roll3_std}_t = \frac{1}{3} \sum_{i=t-2}^t (y_i - \text{roll3_mean}_t)^2$. Together, these engineered features enhance the model's ability to detect both regular temporal cycles and recent fluctuations in electricity demand [46]. To model exogenous influences, meteorological variables from four weather stations (Jeju-si, Gosan, Seongsan, Seogwipo) are incorporated. Each station contributes $V = 9$ covariates such as ambient temperature (TA), humidity (HM), and solar irradiance (SI), resulting in $4 \times 9 = 36$ raw weather features. To account for temporal delays in environmental influence, we compute both 1-hour and 24-hour lagged versions for each weather variable $x^{(v)}$:

$$x_{\text{lag1},t}^{(v)} = x_{t-1}^{(v)}, \quad x_{\text{lag24},t}^{(v)} = x_{t-24}^{(v)} \quad (6)$$

After all feature transformations, any rows containing NaNs due to lagging or rolling operations are removed to preserve model input consistency. The lag and rolling-window parameters were selected based on domain knowledge and empirical validation. The 1-hour lag (lag_1) captures short-term autocorrelation between consecutive hours, while the 24-hour lag (lag_{24}) reflects the strong daily periodicity inherent in electricity load patterns. The 3-hour rolling statistics were introduced to smooth short-term fluctuations while preserving local demand dynamics. Preliminary sensitivity analysis indicated that these settings provided stable validation

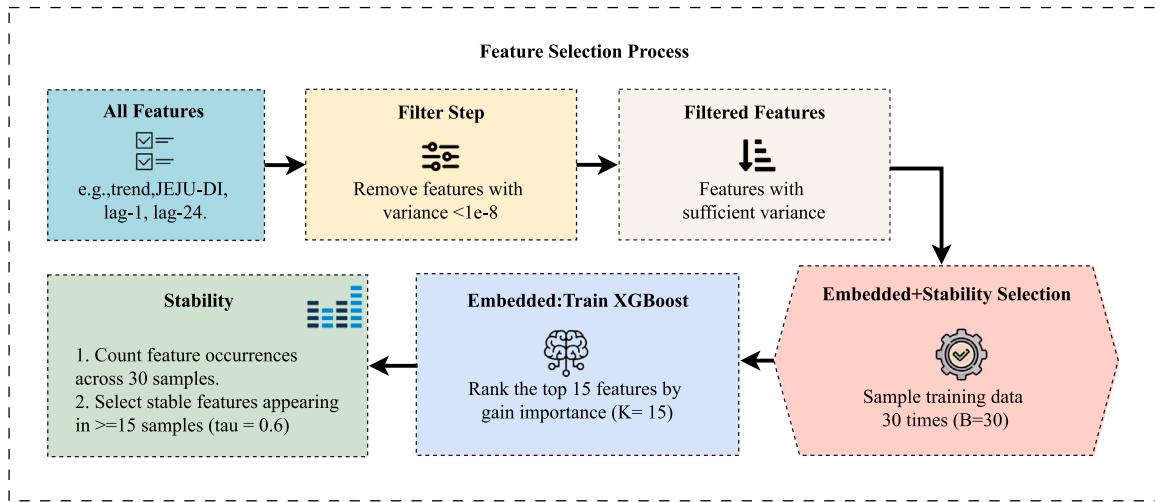


Fig. 3. Hybrid embedded + stability feature selection process: Low-variance features are removed, followed by 30 bootstrap XGBoost runs ranking the top 15 features by gain. Features appearing in at least 15 runs ($\tau = 0.6$) are retained as the final stable subset.

performance while maintaining model simplicity. Moreover, the embedded–stability feature selection process further ensured that only statistically robust lag features were retained in the final model. Finally, the dataset is partitioned chronologically into training and testing sets:

$$D_{\text{train}} = \{(x_t, y_t) \mid t < \text{Jan 1, 2019}\}, \quad D_{\text{test}} = \{(x_t, y_t) \mid t \geq \text{Jan 1, 2019}\}$$

This forward-looking split mimics the conditions of real-world deployment, where forecasting models must generalize to unseen future data. Following this partitioning, the training set D_{train} is passed to the feature selection pipeline to identify a compact and informative subset of predictors. This step is crucial to reduce dimensionality, eliminate noisy or redundant features, and improve model interpretability and generalization. In the proposed framework, Prophet is applied in a univariate manner to decompose the load series into trend and seasonal components. The 36 meteorological covariates are not incorporated as external regressors within the Prophet model. Instead, they are integrated in the subsequent XGBoost regression stage together with temporal and lag features. To mitigate potential multicollinearity and redundancy among weather variables, we employ variance filtering followed by embedded–stability feature selection, ensuring that only robust and consistently important predictors are retained in the final model.

4.2. Embedded+stability feature selection

The feature selection process combines variance filtering with bootstrap-based embedded importance ranking to identify a robust and parsimonious feature subset [47]. The diagram in Fig. 3 illustrates the multi-stage selection pipeline: starting from all engineered features (trend, calendar, lags, weather), we first filter out near-constant predictors. The remaining features undergo 30 bootstrapped fits of XGBoost, each time ranking the top 15 by gain importance. Finally, only those features that appear in at least 60% of the samples ($\tau = 0.6$) are retained as the stable subset for downstream modeling.

Features with variance below a threshold of $\text{Var}(X_j) < 1 \times 10^{-8}$ are removed using the VarianceThreshold tool. The variance $\text{Var}(X_j)$ for each feature X_j is computed as follows:

$$\text{Var}(X_j) = \frac{1}{n-1} \sum_{i=1}^n (X_{ij} - \bar{X}_j)^2 \quad (7)$$

where X_{ij} is the value of feature j for observation i , $\bar{X}_j$ is the sample mean across all n observations, and the summation computes the squared deviations from the mean. Features failing to exceed this threshold are excluded, yielding a refined set of filtered features (filtered_feats) for subsequent processing. The hybrid embedded+stability selection process trains XGBoost $B = 30$ times, each time drawing a bootstrapped sample with replacement from the data. In the Embedded Step, each sample b becomes a training iteration where XGBoost performs regression on the filtered features with labels y . The model optimizes an objective function balancing regularization and loss:

$$\mathcal{L} = \sum_{i=1}^n l(y_i, \hat{y}_i) + \Omega(f) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (8)$$

penalizes model complexity based on the number of trees T , leaf weights w_j , and regularization hyperparameters γ and λ [48]. Feature importance is quantified using gain, which measures the improvement in loss from a particular split:

$$\text{Gain}(j) = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \quad (9)$$

where G_L , G_R and H_L , H_R are the first- and second-order gradients of the loss for the left and right child nodes, respectively. From each bootstrap sample, the top $K = 15$ features with the highest gain are selected [49]. The stability step then counts the frequency of each feature j among the top- K across all 30 bootstrap iterations, recording $C(j)$ as the occurrence count. A feature is retained in the final subset if it meets the stability threshold:

$$\text{Selected if } C(j) \geq \tau \cdot B \quad (10)$$

where $\tau = 0.6$ and $B = 30$, meaning features must appear in the top- K in at least 18 of the 30 iterations. This criterion ensures retention of only consistently important predictors across different data resamples. The resulting feature set, `selected_feats`, comprises 14 predictors that are both consistently important and robust to data resampling. These features form the foundation for the downstream XGBoost regression model [9]. Formally, let B denote the number of bootstrap iterations, K the number of top-ranked features selected per iteration, and

$$F = \{1, 2, \dots, p\} \quad (11)$$

the full set of candidate features. For each bootstrap iteration $b = 1, \dots, B$, a bootstrap sample $X^{(b)}$ is drawn from the training data, and an XGBoost model is trained. Features are ranked using the gain-based importance metric. The subset of selected features in iteration b , denoted by $S_b \subset F$, is defined as

$$S_b = \text{TopK}(\text{Gain}(X^{(b)})). \quad (12)$$

The selection frequency of feature $j \in F$ across all bootstrap iterations is computed as

$$C(j) = \sum_{b=1}^B \mathbf{1}(j \in S_b), \quad (13)$$

where $\mathbf{1}(\cdot)$ denotes the indicator function, and a feature is retained in the final stable subset if its selection frequency satisfies $C(j) \geq \tau B$, where $\tau \in (0, 1]$ is the predefined stability threshold; accordingly, the final selected feature set is defined as,

$$S^* = \{j \in F \mid C(j) \geq \tau B\}. \quad (14)$$

Algorithm 1 Embedded-Stability Feature Selection

Require: Training data X , number of bootstraps B , top- K threshold K , stability threshold τ

Ensure: Stable feature subset S^*

```

1: for  $b = 1$  to  $B$  do
2:   Draw bootstrap sample  $X^{(b)}$ 
3:   Train XGBoost on  $X^{(b)}$ 
4:   Compute gain-based feature importance
5:   Select top- $K$  features  $\rightarrow S_b$ 
6: end for
7: Compute selection frequency  $C(j)$ 
8:  $S^* = \{j \in F \mid C(j) \geq \tau B\}$ 
9: return  $S^*$ 
```

The complete embedded-stability feature selection procedure is summarized in Algorithm 1. This process produces a robust and stable feature subset suitable for reliable downstream modeling. The hyperparameters were selected based on empirical sensitivity analysis to ensure stability while maintaining computational efficiency. Preliminary experiments indicated that feature selection frequencies converged beyond approximately 25 bootstrap iterations; therefore, $B = 30$ was chosen as a stable yet computationally efficient setting. Similarly, values of K in the range 12–18 yielded comparable validation performance, with $K = 15$ providing the most stable and parsimonious subset. The stability threshold $\tau = 0.6$ was adopted to retain only features consistently selected across the majority of bootstrap samples.

4.2.1. Feature selection outcomes

The Feature Selection outcomes reflect the practical application of the Filter Step and Embedded+Stability Selection. The initial variance filtering successfully reduced the feature space by removing near-constant predictors, setting the stage for the hybrid selection process. The subsequent 30 bootstrapped XGBoost iterations, each ranking the top 15 features by gain, allowed for a robust assessment of feature importance, while the stability criterion ($\tau = 0.6$) ensured only consistently significant features were retained.

The efficacy of this approach is evidenced in Fig. 4, which illustrates the feature selection outcome. The left panel displays absolute counts: out of 111 total engineered features, 14 (12.6%) were retained by the hybrid embedded+stability method, while 97

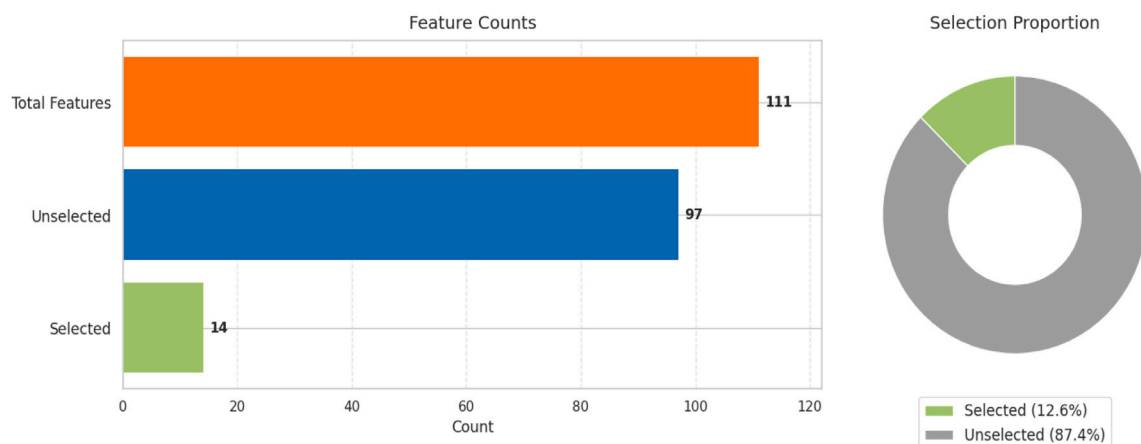


Fig. 4. Outcome of the hybrid feature selection process, where 14 of 111 features (12.6%) were identified as stable and informative. The remaining 97 features (87.4%) were excluded due to low variance or inconsistent selection across bootstrap iterations.

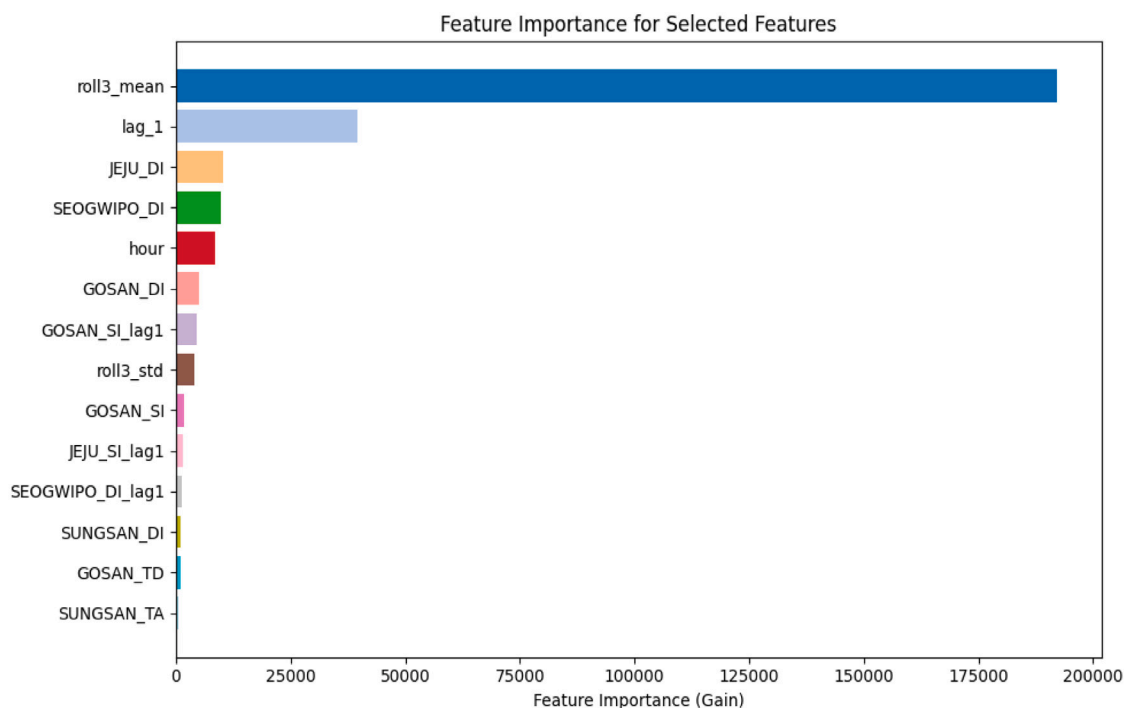


Fig. 5. Gain-based feature importance of the 14 predictors selected through the hybrid embedded+stability feature selection process.

(87.4%) were discarded. This significant reduction underscores the filter's role in eliminating low-variance features and the stability step's success in identifying a parsimonious subset. The right donut chart complements this by highlighting the same proportions in relative terms, visually emphasizing the efficiency and selectivity of the chosen feature set for subsequent modeling. Further insight into the selected features is provided by Fig. 5, which presents a bar chart of the relative "gain" importance within the final XGBoost model. This metric, a direct outcome of the Embedded Step's gain calculation, quantifies every feature's role in reducing the model's training error. Longer bars indicate greater total split-gain across all trees, revealing that the 3-period rolling mean (`roll3_mean`) emerges as the most influential predictor, followed by the 1-hour lag (`lag_1`). These findings align with the Embedded Step's focus on high-gain features, while weather indices and calendar variables, though contributing smaller gains, add meaningful incremental improvements, reflecting the stability criterion's role in retaining diverse yet relevant predictors.

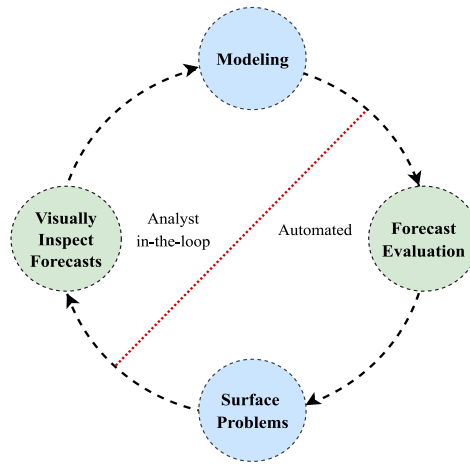


Fig. 6. Prophet forecasting workflow. The cycle combines automated modeling and evaluation with human inspection to identify issues and refine forecasts iteratively.

4.3. Prophet model

Prophet, introduced by Meta's (formerly Facebook) Data Science team [44], is a statistical model for time series forecasting. It is particularly effective in handling real-world complexities such as multiple seasonalities, missing observations, outliers, and sudden trend changes [50]. Unlike deep learning approaches, Prophet is grounded in a classical statistical framework, implemented through Bayesian inference [51], as depicted in Fig. 6. The model represents a time series as the sum of four components: a trend function $g(t)$, a seasonal component $s(t)$, holiday effects $h(t)$, and a stochastic error term ϵ_t that accounts for noise [52].

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t \quad (15)$$

The trend component, denoted as $g(t)$, captures the non-periodic, long-term evolution of the time series. Prophet accommodates both linear and logistic growth trends to model such changes effectively [52]. A frequently employed form is the logistic growth function as show in Eq. (16):

$$g(t) = \frac{C}{1 + \exp(-k(t - m))} \quad (16)$$

Here, C represents the carrying capacity, k denotes the growth rate, and m is the offset parameter that controls the horizontal shift of the curve. This formulation allows the model to capture saturating growth patterns commonly observed in real-world time series. To account for abrupt changes in the trend, Prophet incorporates a piecewise logistic growth model with designated changepoints [53].

$$g(t) = \frac{C(t)}{1 + \exp(-(k + \mathbf{a}(t)^\top \boldsymbol{\delta})(t - (m + \mathbf{a}(t)^\top \boldsymbol{\gamma})))} \quad (17)$$

In Eq. (17), $\mathbf{a}(t)$ is a binary indicator vector that activates when the time t surpasses a changepoint. The vector $\boldsymbol{\delta}$ modulates the growth rate, while $\boldsymbol{\gamma}$ adjusts the offset at those changepoints. To prevent overfitting, Prophet applies a Laplace before regularizing the values of $\boldsymbol{\delta}$.

$$\delta_j \sim \text{Laplace}(0, \tau) \quad (18)$$

This prior encourages sparsity, ensuring that only substantial changes in the trend are captured. As a result, the model remains robust to noise and avoids overfitting while effectively modeling structural shifts. The seasonality term, denoted as $s(t)$, models recurring patterns in the time series [54]. These patterns may follow daily, weekly, yearly, or other cyclical intervals, enabling the model to account for regular temporal fluctuations.

$$s(t) = \sum_{n=1}^N \left[a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right] \quad (19)$$

In Eq. (19), P denotes the known period of the seasonal cycle (e.g., 365.25 for yearly seasonality), N is the Fourier order controlling the number of sine and cosine terms, and a_n, b_n are learned coefficients. This Fourier-based formulation enables Prophet to flexibly capture complex and potentially nonlinear seasonal patterns. The holiday term, $h(t)$, accounts for the effects of holidays or special events that may cause temporary but significant deviations in the time series. These effects are modeled as:

$$h(t) = \sum_j \kappa_j Z_j(t) \quad (20)$$

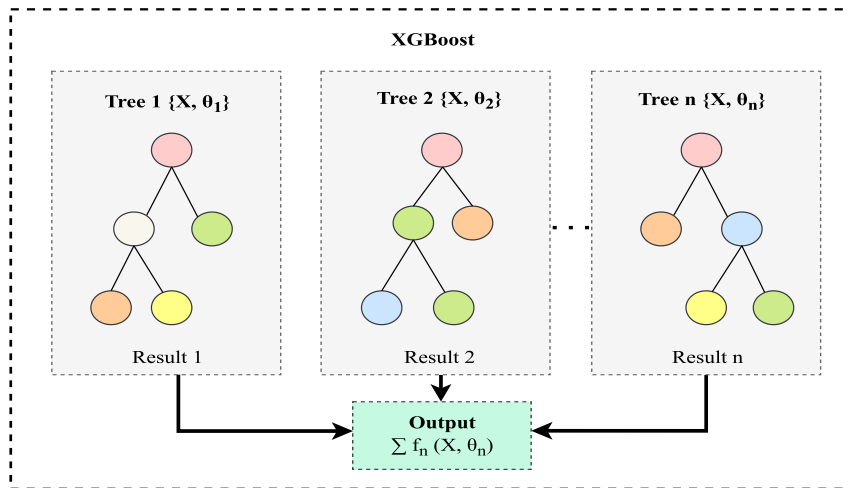


Fig. 7. Illustration of XGBoost architecture. It is a stage-wise ensemble of regression trees. Each tree predicts residuals, and the final output is the sum of weighted leaf contributions.

Here, $Z_j(t)$ is an indicator function that equals 1 if day t corresponds to holiday j , and 0 otherwise. The parameter κ_j quantifies the estimated impact of holiday j on the time series. This component enhances the model's ability to capture irregular events that lead to temporary spikes or drops in the data [55]. The error term, ϵ_t , represents random noise and unexplained variation not captured by the trend, seasonality, or holiday components. It is assumed to be frequently distributed and reflects the inherent uncertainty and stochastic nature of real-world data. This component is essential for generating probabilistic forecasts, allowing the model to output a distribution of potential future outcomes rather than a single deterministic prediction.

4.4. XGBoost (Extreme gradient boosting)

Extreme Gradient Boosting (XGBoost) is a scalable and highly effective gradient boosting decision tree algorithm, widely used in machine learning tasks due to its ability to optimize both accuracy and computational speed [56]. It is designed to handle large datasets with high efficiency, making it ideal for real-world applications [57]. The model learns in an additive, stage-wise fashion, where each tree is trained to correct the errors made by the previous one, thus improving predictive performance incrementally. XGBoost utilizes second-order derivatives to optimize the loss function, which enables it to capture complex patterns in the data more effectively than traditional gradient boosting methods [58]. Additionally, XGBoost incorporates regularization techniques [59], including L1 (Lasso) and L2 (Ridge) penalties, to mitigate overfitting and improve generalization performance on unseen data, as illustrated in Fig. 7.

Given a dataset with n instances $\{(x_i, y_i)\}_{i=1}^n$, the predicted output at the t th iteration of the boosting process is defined as follows in Eq. (21):

$$\hat{y}_i^{(t)} = \sum_{k=1}^t f_k(x_i), \quad f_k \in \mathcal{F} \quad (21)$$

where, f_k are regression trees from the function space $\mathcal{F} = \{f(x) = w_{q(x)}\}$, and $q(x)$ maps an input to the corresponding leaf node with weight w . The overall objective function to minimize at each iteration is shown in (22):

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^t \Omega(f_k) \quad (22)$$

where, l is a differentiable convex loss function (e.g., squared error), and the regularization term is defined as:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \quad (23)$$

Here, T is the number of leaves, and γ, λ are regularization parameters. To optimize the objective efficiently, XGBoost uses a second-order Taylor expansion [60] to approximate the loss as shown in Eq. (24):

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_i(x_i) + \frac{1}{2} h_i f_i(x_i)^2 \right] + \Omega(f_t) \quad (24)$$

where:

$$g_i = \frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i}, \quad h_i = \frac{\partial^2 l(y_i, \hat{y}_i)}{\partial \hat{y}_i^2} \quad (25)$$

Table 6

Represents the exact compute environment in which all of your model training and evaluation experiments were carried out.

Component	Specification
CPU	13th Gen Intel®Core™i7-13700 @ 2.10 GHz
GPU	NVIDIA GeForce RTX 4060 (8 GB dedicated)
System	Windows 11 Home
RAM	16.0 GB DDR5 @ 5600 MHz
IDE	Jupyter Notebook

represent the loss's first and second derivatives concerning the prediction. For a given tree, let the set of data points falling into leaf j be I_j . Define the sums of gradients and Hessians in that leaf as:

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i \quad (26)$$

The optimal weight for each leaf j and the corresponding regularized loss of the tree are defined in Eqs. (27) and (28), respectively. In XGBoost, gradient-based optimization assigns optimal weights to leaf nodes to minimize the regularized loss. Each weight w_j^* is derived from the first-order (gradient) and second-order (Hessian) statistics, ensuring numerically stable updates.

$$w_j^* = -\frac{G_j}{H_j + \lambda} \quad (27)$$

$$\mathcal{L}_{\text{leaf}} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T \quad (28)$$

The total tree loss aggregates these leaf-wise contributions, effectively balancing predictive accuracy with model complexity to mitigate overfitting. When deciding where to split a node, XGBoost evaluates the gain from a potential split as following Eq. (29):

$$\mathcal{L}_{\text{split}} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \quad (29)$$

where G_L, H_L and G_R, H_R are the gradient and Hessian sums for the left and right child nodes, respectively. XGBoost supports efficient handling of sparse data, incorporates built-in regularization to reduce overfitting [61], and provides feature importance scores for model interpretation. Its flexibility and performance make it a leading choice for many structured data tasks [62].

4.5. Model training and evaluation

To ensure the reproducibility and consistency of the experimental results, all model training, feature selection, and evaluation procedures were conducted on a standardized computing setup. Table 6 details the hardware and software specifications of the system used throughout the study. To ensure a fair and consistent comparison, we employed four standard evaluation metrics [63], each capturing different aspects of forecasting accuracy:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (30)$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad (31)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (32)$$

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|. \quad (33)$$

The forecasting performance is assessed using four widely adopted metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE), as presented in Eqs. (22)–(25). In these formulations, y_i and $\hat{y}_i$ represent the actual and predicted values, respectively, while n denotes the number of observations. MAE quantifies the average magnitude of absolute errors, whereas MSE penalizes larger deviations more heavily by squaring them. RMSE, obtained as the square root of MSE, preserves the original units of the target variable and accentuates substantial errors. MAPE expresses the error as a percentage, enabling scale-independent comparison, though it is sensitive to small actual values. Collectively, these metrics provide a comprehensive evaluation of model accuracy and robustness. These metrics were computed on the test set (2019–2020), providing a balanced assessment of average error magnitude (MAE, MAPE) and sensitivity to large deviations (MSE, RMSE).

Table 7

Overall test-set performance of the ProphetBoost model before and after applying the proposed hybrid feature-selection strategy.

Metric	Without feature selection	With feature selection
MAE↓	6.44	5.36
MSE↓	113.48	85.60
RMSE↓	10.65	9.47
MAPE (%)↓	0.92	0.77

Table 8

Daily and monthly forecasting performance of the ProphetBoost model on the 2019–2020 test period. The results highlight temporal variability in load predictability, with weekends and early-spring months exhibiting the lowest error levels.

Time period	MAE↓	MSE↓	RMSE↓	MAPE (%)↓
<i>Daily Metrics</i>				
Monday	5.46911	76.5871	8.75141	0.787327
Tuesday	6.08908	114.187	10.6858	0.865476
Wednesday	5.80371	92.8051	9.63354	0.833226
Thursday	5.83578	101.841	10.0916	0.858346
Friday	5.68422	91.7073	9.57640	0.805920
Saturday	5.07509	60.4363	7.77408	0.747680
Sunday	5.74737	75.0371	8.66240	0.823289
<i>Monthly Metrics</i>				
January	5.12855	64.1707	8.01066	0.751638
February	5.36753	71.2805	8.44278	0.776285
March	4.92101	59.9113	7.74024	0.702120
April	5.69471	85.2099	9.23092	0.807662
May	6.22903	134.699	11.60600	0.916602
June	5.93373	120.598	10.98170	0.858456
July	5.57686	85.2152	9.23121	0.811080
August	5.98962	91.5445	9.56789	0.852086
September	5.57613	65.3150	8.08177	0.805526
October	5.45380	82.9426	9.10728	0.798967
November	5.91268	85.3523	9.23863	0.839741
December	6.32106	106.149	10.30290	0.891278

5. Experimental results

In our analysis, we evaluate the impact of integrating a hybrid feature-selection step into the XGBoost pipeline for load forecasting. To evaluate the effectiveness of the proposed hybrid forecasting framework, we conducted a series of experiments comparing the model's performance with and without the embedded + stability feature selection step, as shown in Table 7.

The Mean Squared Error (MSE) drops significantly from 113.48 to 85.60 MW, reflecting a clear reduction in large deviation errors. Likewise, the Root Mean Squared Error (RMSE) improves from 10.65 MW to 9.47 MW, indicating better generalization on unseen data. The Mean Absolute Percentage Error (MAPE) is also reduced from 0.92% to 0.77%, demonstrating a noticeable gain in relative forecasting accuracy. Collectively, these improvements validate the effectiveness of the proposed feature-selection step in eliminating noisy or redundant predictors, thereby simplifying the model while enhancing its predictive performance. To further evaluate the temporal robustness of the framework, Table 8 reports forecasting performance across different days of the week and months of the year. In terms of daily behavior, the model performs most accurately on Saturdays, achieving the lowest MAE of about 5.08 MW, RMSE of 7.77 MW, and MAPE of 0.75%. By contrast, Tuesdays yield the highest daily error, with MAE of 6.09 MW, RMSE of 10.69 MW, and MAPE of 0.87%, likely reflecting mid-week demand peaks and greater variability. These findings suggest that the model is particularly effective during weekends, when load patterns tend to be more stable than on weekdays. On a monthly scale, March stands out as the most accurately forecasted period, with MAE of 4.92 MW, RMSE of 7.74 MW, and MAPE of 0.70%. This indicates strong alignment with early-spring load dynamics, which are generally less volatile than transitional or peak-demand seasons. Conversely, May emerges as the most challenging month, registering the highest error values across all metrics (MAE = 6.23 MW, RMSE = 11.61 MW, MAPE = 0.92%). This behavior may be attributed to increased variability in electricity usage during late spring, driven by fluctuating weather conditions and the onset of cooling demand.

Diagnostics of the Final XGBoost Forecasting Model are shown in Fig. 8, Panel A plots training and validation MAE over 1000 boosting rounds (early-stopped at 1000), showing rapid decline and stable convergence. Panel B overlays actual versus predicted hourly loads for 2019–2020, demonstrating tight tracking of daily peaks and valleys. Panel C's residual density is sharply centered at zero, with most errors within ± 20 MW, indicating low bias and variance. Panel D presents the top-10 gain-based feature importances led by `roll3_mean`, `lag_1`, and key weather covariates, highlighting the model's reliance on recent load dynamics and localized meteorological signals. The influence and error structure of meteorological covariates are examined in each row of Fig. 9. In addition

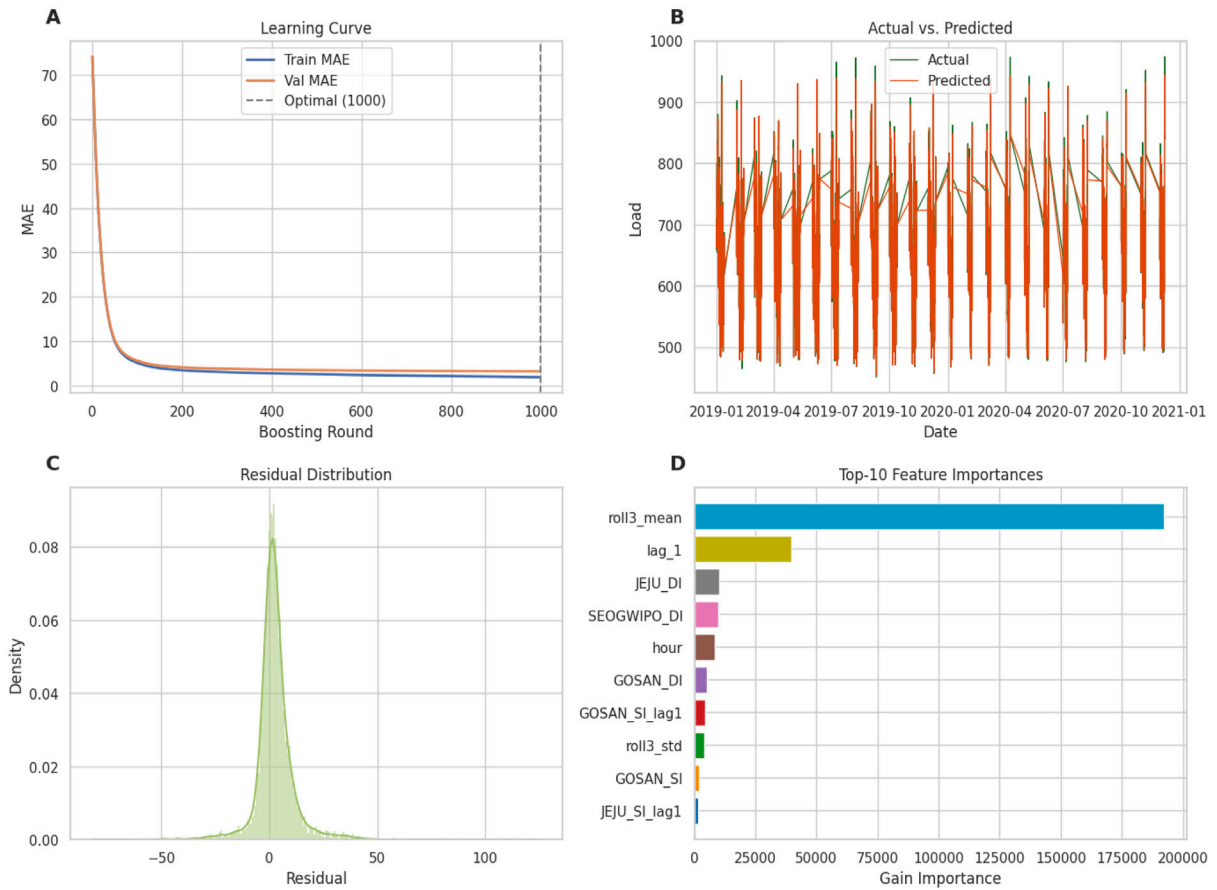


Fig. 8. XGBoost model performance and interpretation summary. (A) Learning curve showing convergence of training and validation MAE over boosting rounds. (B) Actual vs. predicted load values across the test period, demonstrating strong alignment. (C) Residual distribution indicating minimal bias and near-normal error spread. (D) Top-10 feature importances (gain), highlighting `roll3_mean` and `lag_1` as dominant predictors.

to a combined residual plot, Row 1 displays the partial dependence of the dew-point index (DI) across the four stations (Jeju, Gosan, Sungsan, and Seogwipo). It shows that a higher DI at Seogwipo significantly lowers the predicted load before increasing once more, with residuals for all stations closely clustered around zero. Gosan's dew-point temperature (TD), which is the subject of Row 2, has a partial influence that varies between 5 and 15 degrees Celsius. Its residuals are symmetrically distributed, suggesting that it makes unbiased forecasts throughout its range. Gosan's sun irradiance (SI) is seen in Row 3, where the residuals lack a consistent pattern and the model response drops off dramatically at low SI values. Sungsan's air temperature (TA) is shown in Row 4 with equally distributed residuals and a U-shaped partial dependence (lower expected load in mid-range temperatures).

6. Performance evaluation and feature analysis

This section evaluates the performance of the proposed ProphetBoost (Prophet-XGBoost) model for short-term load forecasting and compares it against a range of baseline methods. We first summarize overall forecasting accuracy and the impact of feature selection, then compare ProphetBoost to state-of-the-art benchmark models under a unified hyperparameter optimization protocol. Finally, we analyze computational complexity, statistical significance, and the contribution of the selected features using multiple interpretability tools.

6.1. Benchmark model comparison

To comprehensively evaluate the effectiveness of ProphetBoost, we benchmarked it against a diverse suite of forecasting models spanning multiple methodological paradigms. These include statistical hybrids (ARIMA-LSTM), lightweight linear architectures (DLinear), deep feed-forward and recurrent models (CNN-ANN, CNN-LSTM, Deep RVFL), probabilistic recurrent models (DeepAR), and attention-based architectures (Transformer, Temporal Fusion Transformer (TFT), and Autoformer). This selection ensures coverage of classical, ensemble, deep learning, and state-of-the-art Transformer-based approaches. All models were evaluated on the

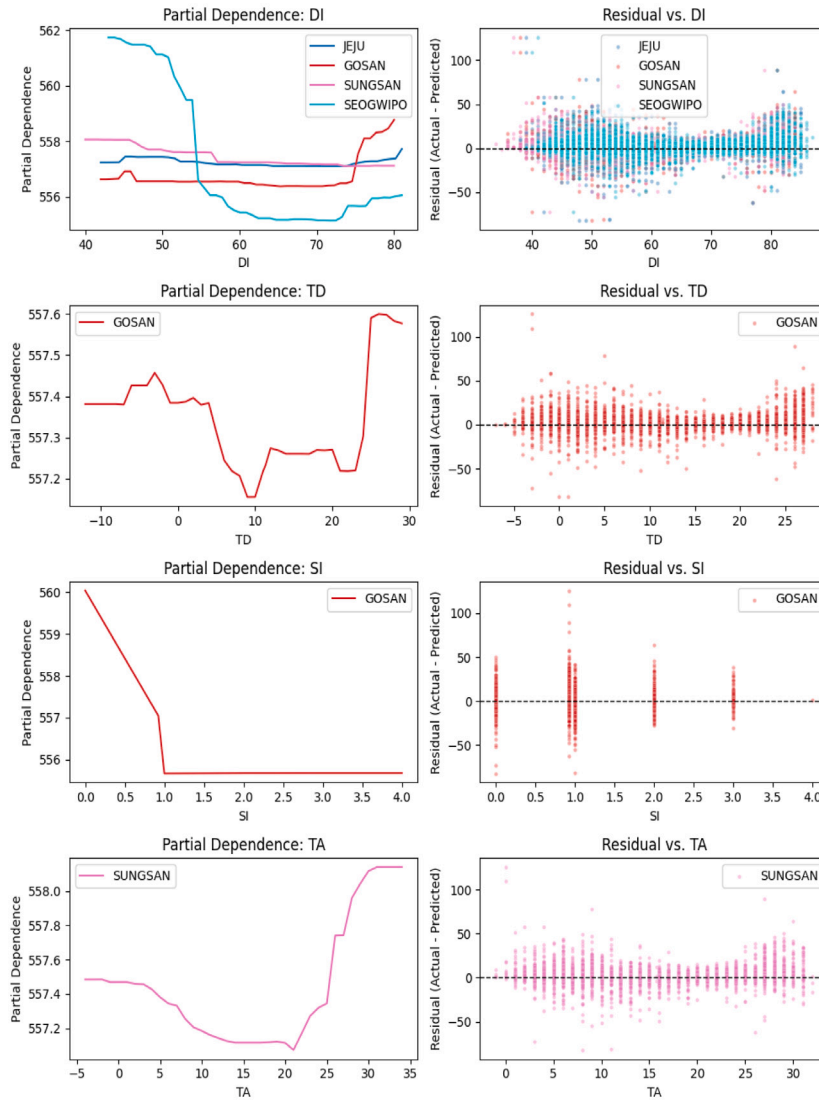


Fig. 9. Partial dependence and residual analysis of key weather-based predictors across different stations. This figure visualizes how selected weather features influence electricity load predictions.

2019–2020 test dataset using four widely adopted performance metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE). A comprehensive comparison is presented in Table 9, which reports results both with and without the proposed embedded-stability feature selection (FS) framework.

The comparative results in Table 9 reveal substantial performance differences across forecasting paradigms. The ARIMA–LSTM hybrid exhibits the weakest accuracy (MAE = 126.56 MW, RMSE = 146.92 MW without FS), indicating that residual learning around a linear ARIMA backbone is insufficient to model highly nonlinear and volatile load dynamics. While CNN–LSTM and CNN–ANN significantly improve over ARIMA–LSTM, their performance remains inferior to more advanced architectures in terms of both MAE and MAPE. DLinear provides a notable improvement over purely statistical hybrids, especially after feature selection (MAE = 16.85 MW), yet its linear structure limits its ability to capture complex temporal dependencies fully. Among the deep learning and attention-based models, Deep RVFL, Transformer, DeepAR, Autoformer, and TFT achieve increasingly competitive results. In particular, TFT (MAE = 9.74 MW without FS; 8.92 MW with FS) and Autoformer (MAE = 10.38 MW without FS; 9.61 MW with FS) demonstrate strong modeling capacity for long-range dependencies, while DeepAR offers stable probabilistic sequence modeling performance (MAE = 11.06 MW without FS; 10.14 MW with FS). Despite their architectural sophistication, these models consistently underperform ProphetBoost across all metrics. ProphetBoost achieves the best overall results, attaining MAE = 6.44 MW and RMSE = 10.65 MW without FS, and further improving to MAE = 5.36 MW, MSE = 85.60 MW², RMSE = 9.25 MW, and MAPE = 0.77% with FS. These findings highlight the advantage of integrating Prophet’s structured temporal decomposition with XGBoost’s nonlinear

Table 9

Performance comparison of all benchmark models with and without feature selection (FS).

Model	Without FS				With FS			
	MAE↓	MSE↓	RMSE↓	MAPE (%)↓	MAE↓	MSE↓	RMSE↓	MAPE (%)↓
ARIMA-LSTM [64]	126.56	21 585.01	146.92	21.00	124.77	21 055.21	145.10	20.78
DLinear [65]	80.06	10 427.29	102.11	12.12	16.85	672.42	25.93	2.52
CNN-ANN [66]	36.76	2346.21	48.44	5.54	23.62	1053.72	32.46	3.57
Deep RVFL [67]	19.27	720.89	26.85	2.84	14.86	531.01	23.04	2.24
CNN-LSTM [68]	13.36	408.71	20.22	1.98	10.80	329.16	18.14	1.59
Transformer [69]	11.50	323.37	17.98	1.72	10.44	305.15	17.47	1.56
DeepAR [70]	11.06	321.94	17.94	1.69	10.14	293.88	17.14	1.53
Autoformer [38]	10.38	298.72	17.28	1.61	9.61	271.35	16.47	1.44
TFT [37]	9.74	276.58	16.63	1.51	8.92	248.41	15.76	1.37
ProphetBoost	6.44	113.48	10.65	0.96	5.36	85.60	9.25	0.77

Table 10

Hyperparameter search space and selected values for all benchmark models. ProphetBoost is tuned via time-series cross-validation (xgb.cv), while deep-learning and linear baselines are optimized using Optuna. CNN-based and ARIMA-LSTM configurations are selected for stable training performance.

Model	Hyper.	Search Space	Final Value	Model	Hyper.	Search space	Final Value
ProphetBoost	learning_rate	[0.01–0.30]	0.05	Deep RVFL	n_models	{3–7}	5
	max_depth	{3,4,5,6,7,8}	6		hidden_dim	128–512	256
	subsample	[0.60–1.00]	0.8		num_layers	{2,3,4,5}	3
	colsample_bytree	[0.60–1.00]	0.8		learning_rate	10^{-4} – 10^{-2}	0.001
	lambda (L2)	[0.01–10]	CV-selected		batch_size	{16,32,64}	32
	alpha (L1)	[0.01–10]	CV-selected		epochs	fixed	20
	num_boost_round	1–1000	best CV round	CNN-LSTM	conv_channels	{32,64,128}	64
DLinear	learning_rate	10^{-4} – 10^{-2}	0.004577		hidden_dim	{64,128,256}	128
	batch_size	{16,32,64}	16		num_layers	{1,2,3}	1
	epochs	fixed	50		learning_rate	10^{-4} – 10^{-2}	0.001
Transformer	d_model	{32,64,128}	32		batch_size	{32,64}	64
	num_heads	{1,2,4,8}	8		epochs	{20–100}	50
	num_layers	{1,2,3}	2	CNN-ANN	conv_channels	{32,64,128}	64
	dim_feedforward	{64,128,256}	256		mlp_hidden_dim	{64,128,256}	128
	learning_rate	10^{-4} – 10^{-2}	5.34e–4		learning_rate	10^{-4} – 10^{-2}	0.001
	batch_size	{16,32,64}	32		batch_size	{32,64}	64
ARIMA-LSTM	epochs	fixed	50		epochs	{20–100}	50
	ARIMA order	(p,d,q)	(5,1,0)	Autoformer	d_model	{32,64,128}	64
	hidden_dim	{32,64,128}	64		n_heads	{2,4,8}	4
	num_layers	{1,2,3}	1		e_layers	{1,2,3}	2
	learning_rate	10^{-4} – 10^{-2}	0.001		d_ff	{64,128,256}	128
	batch_size	{32,64}	64		learning_rate	10^{-4} – 10^{-3}	5e-4
	epochs	{20–100}	50		batch_size	{16,32,64}	32
TFT	hidden_size	{16,32,64}	32	DeepAR	epochs	fixed	50
	lstm_layers	{1,2,3}	2		hidden_size	{20,40,60}	40
	attention_heads	{2,4,8}	4		num_layers	{1,2,3}	2
	dropout	[0.1–0.5]	0.2		dropout	[0.1–0.5]	0.1
	learning_rate	10^{-4} – 10^{-3}	5e-4		learning_rate	10^{-4} – 10^{-3}	1e-3
	batch_size	{16,32,64}	32		batch_size	{16,32,64}	32
	epochs	fixed	50		epochs	fixed	50

ensemble learning, enabling the model to effectively capture both global seasonal patterns and fine-grained local load variations while maintaining superior predictive accuracy.

To ensure a fair and consistent comparison, we employ a unified, leakage-free hyperparameter optimization (HPO) protocol across ProphetBoost and all benchmark models, including Transformer-based architectures (Transformer, TFT, Autoformer), recurrent models (DeepAR, CNN-LSTM, ARIMA-LSTM), linear models (DLinear), and ensemble-based approaches (Deep RVFL). Neural and linear baselines are optimized using Optuna with a Tree-structured Parzen Estimator (TPE) sampler, while the XGBoost component of ProphetBoost is tuned via 5-fold time-series cross-validation (xgb.cv) with early stopping. Search spaces cover architectural parameters (e.g., hidden size, number of layers, attention heads), optimization settings (learning rate, batch size), and regularization terms, with only valid configurations permitted. All deep-learning models are trained using a 24-hour lookback

Table 11

Comparison of computational cost for all models without (top) and with (bottom) feature selection. Only outer borders are displayed for a clean, journal-ready layout.

Without Feature Selection					
Model	Train (s)	Infer (s)	Infer/Sample (ms)	Size (MB)	GPU (MB)
ProphetBoost	19.260	0.006	0.0010	2.41	50.78
Transformer	227.692	0.454	0.0795	8.35	262.61
TFT	270.332	0.616	0.1078	7.77	183.47
Autoformer	308.599	0.432	0.0756	8.30	307.70
DeepAR	201.868	0.522	0.0914	7.59	193.32
DLinear	59.900	0.110	0.0193	4.00	73.47
Deep RVFL	29.636	0.065	0.0112	9.95	83.64
CNN-LSTM	175.425	0.441	0.0773	6.88	179.44
CNN-ANN	46.586	0.096	0.0168	7.08	94.67
ARIMA-LSTM	144.992	0.555	0.0972	5.16	155.82
With Feature Selection					
Model	Train (s)	Infer (s)	Infer/Sample (ms)	Size (MB)	GPU (MB)
ProphetBoost	12.427	0.008	0.0014	2.36	50.78
Transformer	248.508	0.324	0.0567	8.25	261.52
TFT	201.252	0.464	0.0812	7.67	181.32
Autoformer	273.298	0.444	0.0778	8.20	307.61
DeepAR	196.911	0.468	0.0819	7.20	185.85
DLinear	52.686	0.067	0.0116	0.48	53.80
Deep RVFL	27.305	0.060	0.0104	9.56	82.45
CNN-LSTM	162.603	0.395	0.0691	6.73	178.10
CNN-ANN	40.244	0.071	0.0124	6.93	93.36
ARIMA-LSTM	138.936	0.539	0.0944	5.16	155.82

Table 12

Statistical significance analysis comparing ProphetBoost with all baseline models using the Wilcoxon signed-rank test at significance levels $\alpha = 0.025$ and $\alpha = 0.05$, along with the Friedman test for overall model comparison. All pairwise differences are statistically significant ($p < 0.05$), indicating consistent superiority of ProphetBoost across evaluation instances.

Compared models	Wilcoxon signed-rank test		Wilcoxon signed-rank test		Friedman test ($\alpha = 0.05$)
	W	p-Value	W	p-Value	
ProphetBoost vs Transformer	3533383 ^a	0.00000**	3533383 ^a	0.00000**	$F = 16432.5712$ $p = 0.000000$ (Reject H_0)
ProphetBoost vs TFT	3389021 ^a	0.00000**	3389021 ^a	0.00000**	
ProphetBoost vs Autoformer	3314485 ^a	0.00000**	3314485 ^a	0.00000**	
ProphetBoost vs DeepAR	3421157 ^a	0.00000**	3421157 ^a	0.00000**	
ProphetBoost vs DLinear	1626654 ^a	0.00000**	1626654 ^a	0.00000**	
ProphetBoost vs RVFL	2414895 ^a	0.00000**	2414895 ^a	0.00000**	
ProphetBoost vs CNN-LSTM	2748700 ^a	0.00000**	2748700 ^a	0.00000**	
ProphetBoost vs CNN-ANN	763488 ^a	0.00000**	763488 ^a	0.00000**	
ProphetBoost vs ARIMA-LSTM	47427 ^a	0.00000**	47427 ^a	0.00000**	

window and evaluated based on validation MSE during HPO. After selecting optimal hyperparameters, each model is retrained on the full training set and evaluated once on the test set to prevent selection bias. The complete search spaces and selected values are reported in Table 10.

6.2. Computational complexity and statistical significance

Table 11 compares the computational cost of all models in terms of training time, inference time, per-sample latency, model size, and GPU memory usage, before and after feature selection (FS). ProphetBoost exhibits the most efficient profile, reducing training time from 19.260 s to 12.427 s while maintaining extremely low latency (0.0010–0.0014 ms), compact size (~2.4 MB), and minimal GPU usage (~50 MB). In contrast, attention-based models such as Transformer (227.692–248.508 s; ~262 MB), TFT (201.252–270.332 s; ~181–183 MB), and Autoformer (273.298–308.599 s; ~307 MB) incur substantially higher computational costs. DeepAR (196.911–201.868 s; ~186–193 MB), Deep RVFL (27.305–29.636 s; ~82–84 MB), CNN-based models (40.244–175.425 s; ~93–179 MB), and ARIMA-LSTM (138.936–144.992 s; ~156 MB) show moderate complexity. Although FS improves efficiency across models, ProphetBoost remains the most computationally efficient approach.

To verify that the superior performance of ProphetBoost is statistically significant, we conduct Wilcoxon Signed-Rank tests comparing ProphetBoost against each baseline model, along with a Friedman test across all methods. As summarized in Table 12, all pairwise Wilcoxon comparisons yield p -values effectively equal to zero ($p = 0.00000$), indicating that ProphetBoost significantly outperforms Transformer, TFT, Autoformer, DeepAR, DLinear, Deep RVFL, CNN-LSTM, CNN-ANN, and ARIMA-LSTM under both $\alpha = 0.025$ and $\alpha = 0.05$. The Friedman test further rejects the null hypothesis of equal model performance ($F = 16432.5712$,

Table 13

Feature importance scores and selection metrics for the final ProphetBoost model, combining gain-based importance, residual correlation, SHAP values, and stability across bootstrap runs.

Feature	Gain importance	Total % of gain	Corr. (Residual)	Avg SHAP	Sel. Freq.
roll3_mean	192 169.75	45.3%	0.792	64.351	1.00
lag_1	39 648.81	9.4%	0.790	10.898	1.00
JEJU_DI	10 205.36	2.4%	−0.203	0.871	0.60
SEOGWIPO_DI	9879.43	2.3%	−0.209	3.128	0.90
hour	8506.18	2.0%	0.413	10.285	1.00
GOSAN_DI	5093.99	1.2%	−0.207	0.763	0.97
GOSAN_SI_lag1	4523.03	1.1%	−0.053	0.651	1.00
roll3_std	4056.77	1.0%	0.047	4.159	1.00
GOSAN_SI	1811.57	0.4%	0.000	1.174	0.63
JEJU_SI_lag1	1523.70	0.4%	−0.035	0.761	0.93
SEOGWIPO_DI_lag1	1256.33	0.3%	−0.202	2.814	0.60
SUNGSAN_DI	1097.88	0.3%	−0.218	0.455	0.77
GOSAN_TD	969.28	0.2%	−0.237	0.429	0.63
SUNGSAN_TA	550.36	0.1%	−0.231	0.383	0.60

$p = 0.000000$), confirming that the observed performance gains of ProphetBoost are statistically robust and not attributable to random variation.

6.3. Feature importance and SHAP-based interpretation

To understand which predictors drive ProphetBoost's performance, we analyze the 14 selected features using multiple complementary importance metrics: gain importance, total percentage of gain, correlation with residuals, average SHAP values, and selection frequency across 30 bootstrap runs. These statistics are summarized in Table 13. To assess the practical validity of the interpretability results, we further examine whether the SHAP-identified key features align with established operational logic in power systems. Temperature-related variables consistently exhibit high importance scores, particularly during seasonal transition and peak-demand periods. This behavior is consistent with real-world grid dynamics, where sudden temperature increases in summer elevate cooling demand and corresponding load levels. Similarly, calendar-based indicators such as weekend and holiday flags demonstrate negative or moderated contributions relative to weekdays, reflecting reduced industrial and commercial consumption during non-working days.

Moreover, lag-based features (e.g., lag-1 and lag-24) show strong influence, which is consistent with short-term load persistence and daily periodicity commonly observed in electricity demand profiles. These patterns confirm that ProphetBoost not only achieves high predictive accuracy but also learns relationships that are coherent with physical demand drivers and grid operation principles. This alignment between model explanations and domain knowledge enhances the transparency and practical reliability of the proposed framework. The 3-hour rolling mean (roll3_mean) is the dominant feature, contributing 45.3% of total gain and exhibiting the largest average SHAP value. The 1-hour lag (lag_1) contributes an additional 9.4% of gain, confirming the strong autoregressive nature of load dynamics. High selection frequencies (close to 1.0) for roll3_mean, lag_1, hour, and several dew-point and irradiance features indicate that these predictors are consistently informative across bootstrap samples. Correlations with residuals further show that load-history features (roll3_mean, lag_1) are tightly linked with remaining errors, underscoring their central role in the forecasting task. To visualize how these features influence individual predictions, Fig. 10 presents a SHAP beeswarm summary plot for 2000 test observations. Each point represents a single prediction for a given feature, with color encoding the raw feature value (blue = low, red = high) and horizontal position indicating the SHAP contribution magnitude and sign.

The SHAP plot shows that higher values of roll3_mean (red points on the right) drive higher predicted loads, while lower values (blue points) are associated with negative contributions. Similar directional patterns are observed for lag_1 and hour, where mid-day hours and high recent loads push predictions upward. Meteorological variables (e.g., SEOGWIPO_DI, GOSAN_SI) exert more moderate but non-negligible effects, confirming that ProphetBoost effectively integrates both load-history and environmental signals. Together with the partial dependence and residual analyses reported in the preceding figures, these results demonstrate that ProphetBoost not only achieves state-of-the-art forecasting accuracy but also remains interpretable, providing meaningful insights into how temporal and meteorological factors shape short-term electricity demand.

7. Discussion

Electricity demand exhibits structural evolution over time due to economic growth and changing consumption behavior, resulting in non-stationary load patterns. In the Jeju dataset, load levels in 2020 are notably higher than in 2012, reflecting long-term demand growth. The proposed ProphetBoost framework addresses this through Prophet's automatic trend and changepoint modeling, which captures gradual structural shifts, while XGBoost models nonlinear residual dynamics. The strict chronological training and testing protocol ensures evaluation under evolving demand conditions. The strong performance on the 2020 test set indicates that the framework remains robust and adaptive to significantly different load scenarios. This study introduces *ProphetBoost*, a hybrid short-term load forecasting framework that integrates Prophet's additive time-series decomposition with the nonlinear predictive

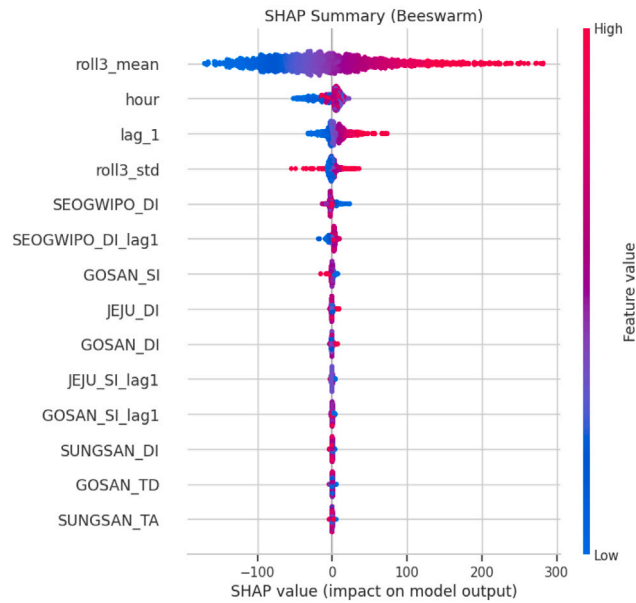


Fig. 10. SHAP beeswarm summary for the Prophet-XGBoost model, illustrating the impact of each selected feature on the model's output across 2000 test observations.

capability of XGBoost, augmented by an XAI-driven embedded-stability feature selection strategy. The empirical results demonstrate that the proposed pipeline simultaneously enhances predictive accuracy and interpretability, addressing two key challenges in energy forecasting. By exploiting eight years of hourly electricity demand data alongside meteorological observations from four geographically distributed weather stations, the model effectively captures both global structures (long-term trends and multiple seasonalities) and local nonlinear dynamics. This framework's two-stage feature selection procedure is a crucial breakthrough, which combines variance thresholding with bootstrap-based embedded ranking using XGBoost's gain importance. This procedure reduced the dimensionality of the feature space by approximately 87% (from 111 to 14 predictors), while improving overall accuracy, with the Mean Absolute Error (MAE) decreasing from 6.44 MW to 5.36 MW. The retained predictors, notably *roll3_mean* and *lag_1*, together with weather-based variables such as dew-point index and solar irradiance, exhibited strong and consistent relevance across resampled models. The variation in forecasting performance across months is primarily attributed to seasonal load dynamics and differing levels of demand volatility. Summer and winter months generally exhibit higher variability due to cooling and heating demands, leading to increased nonlinear behavior and slightly larger forecasting errors. In contrast, transitional months such as March typically have lower average load levels and smoother demand patterns, resulting in smaller absolute errors (e.g., MAE). Additionally, the strength of the weather-load relationship varies seasonally, with stronger temperature sensitivity during peak-demand periods. Therefore, the observed monthly performance differences reflect inherent seasonal characteristics of electricity demand rather than model instability. SHAP-based attribution analysis confirmed their explanatory contribution, aligning with operational and physical intuition. From an application perspective, the interpretability of *ProphetBoost* enhances its suitability for *Building Energy Management Systems* (BEMS). Accurate and transparent forecasts are essential for demand-response programs, HVAC scheduling, and peak-shaving strategies, where operational trust and regulatory compliance require both performance and explainability. Temporal robustness analyses further confirmed that the framework performs more accurately under stable demand conditions, such as weekends and early spring months, while maintaining resilience in more volatile periods. Nevertheless, some limitations remain. Prophet assumes additive decomposition and therefore does not dynamically capture nonlinear interactions between trend and seasonal components. Furthermore, only historical weather data were incorporated as exogenous variables; integrating forecasted meteorological inputs could improve real-time building-level forecasting. Although eight years of data were used to evaluate robustness under long-term structural evolution, the proposed framework does not inherently require extensive historical records. Prophet can effectively capture trends and seasonal patterns once key cycles are observed, and XGBoost performs well under moderate data availability. For newly constructed buildings, the model can be trained using shorter rolling windows and incrementally updated as new data accumulates. Therefore, the framework remains adaptable and applicable in limited-data scenarios. Finally, although XGBoost is computationally efficient for structured data, the full pipeline, including decomposition, lag generation, rolling statistics, and bootstrapping may require optimization for latency-sensitive applications such as edge-deployed BEMS. Overall, the results highlight that *ProphetBoost* provides a scientifically rigorous and practically interpretable solution, bridging advanced machine learning with operational transparency. This positions the framework as a scalable tool for enhancing energy efficiency in buildings and supporting resilient smart grid integration.

8. Conclusion

In this study, we introduced *ProphetBoost*, a hybrid forecasting framework that integrates Prophet's additive decomposition with XGBoost regression, enhanced by an XAI-driven embedded-stability feature selection strategy. By combining the interpretability of time-series decomposition with the nonlinear predictive power of gradient boosting, the framework effectively models complex electricity demand patterns shaped by weather variability, occupant behavior, and renewable energy integration. The two-stage feature selection process achieved substantial dimensionality reduction (87.4%, from 111 to 14 features), retaining only a compact and stable subset of predictors validated through SHAP analysis. This pruning not only enhanced model interpretability but also improved forecasting accuracy, achieving a 16.77% reduction in Mean Absolute Error (MAE) compared to the baseline without feature selection (from 6.44 MW to 5.36 MW). Comprehensive comparative experiments confirmed that *ProphetBoost* consistently outperformed statistical, machine learning, and deep learning baselines across all evaluation metrics. Temporal analysis further demonstrated robustness, with stronger forecasting performance during weekends and early-spring months characterized by more stable demand profiles. These results validate the effectiveness of combining statistical decomposition with gradient boosting for balancing predictive accuracy with interpretability in practical energy forecasting applications. *ProphetBoost* provides transparent, feature-level insights into short-term electricity demand drivers, enabling direct application to HVAC scheduling, demand-response programs, and peak-load management within Building Energy Management Systems (BEMS). The framework's low computational overhead makes it suitable for operational deployment. Limitations of the current work include Prophet's additive decomposition assumption, which may not fully capture nonlinear interactions between trend and seasonal components, and the use of historical weather data as exogenous variables. Future research directions include incorporating forecasted meteorological inputs for multi-step forecasting, integrating real-time smart meter data, and optimizing the pipeline for edge computing environments to support distributed BEMS deployment. Overall, *ProphetBoost* establishes a scientifically rigorous and scalable foundation for interpretable short-term load forecasting, with direct implications for advancing energy efficiency and grid resilience in smart urban environments.

CRedit authorship contribution statement

Nazrul Amin: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review & editing. **Yong-Woon Kim:** Conceptualization, Validation, Visualization, Writing – review & editing. **Yung-Cheol Byun:** Conceptualization, Methodology, Validation, Writing – review & editing, Funding acquisition.

Data & code availability

The dataset and complete ProphetBoost source code are publicly available at [71].

Declaration of Generative AI and AI-assisted technologies in the writing process

The authors used GPT-5.2 to improve the language and readability while preparing this work. Following the use of this tool or service, the author or authors took full responsibility for the publication's content and reviewed and edited it as necessary.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF), South Korea grant funded by the Korea government (MSIT) (No. RS-2024-00405278). This research was also supported by the National Research Foundation of Korea (NRF), South Korea grant funded by the Korea government (MSIT) (RS-2026-25470261).

Data availability

The dataset and complete ProphetBoost source code are publicly available at [71].

References

- [1] Junior MY, Freire RZ, Seman LO, Stefenon SF, Mariani VC, dos Santos Coelho L. Optimized hybrid ensemble learning approaches applied to very short-term load forecasting. *Int J Electr Power Energy Syst* 2024;155:109579. <http://dx.doi.org/10.1016/j.ijepes.2023.109579>.
- [2] Zhao X, Gao W, Qian F, Ge J. Electricity cost comparison of dynamic pricing model based on load forecasting in home energy management system. *Energy* 2021;229:120538. <http://dx.doi.org/10.1016/j.energy.2021.120538>.

- [3] Chen Y, Chen W, Chandra Pal S, Saha A, Chowdhuri I, Adeli B, Janizadeh S, Dineva AA, Wang X, Mosavi A. Evaluation efficiency of hybrid deep learning algorithms with neural network decision tree and boosting methods for predicting groundwater potential. *Geocarto Int* 2022;37(19):5564–84. <http://dx.doi.org/10.1080/10106049.2021.1920635>.
- [4] Sudakov O, Burnaev E, Koroteev D. Driving digital rock towards machine learning: Predicting permeability with gradient boosting and deep neural networks. *Comput Geosci* 2019;127:91–8. <http://dx.doi.org/10.1016/j.cageo.2019.02.002>.
- [5] Hassija V, Chamola V, Mahapatra A, Singal A, Goel D, Huang K, Scardapane S, Spinelli I, Mahmud M, Hussain A. Interpreting black-box models: a review on explainable artificial intelligence. *Cogn Comput* 2024;16(1):45–74. <http://dx.doi.org/10.1007/s12559-023-10179-8>.
- [6] Machlev R, Heistrone L, Perl M, Levy KY, Belikov J, Mannor S, Levron Y. Explainable Artificial Intelligence (XAI) techniques for energy and power systems: Review, challenges and opportunities. *Energy AI* 2022;9:100169. <http://dx.doi.org/10.1016/j.egyai.2022.100169>.
- [7] Chatterjee S, Hazra D, Byun Y-C. GAN-based synthetic time-series data generation for improving prediction of demand for electric vehicles. *Expert Syst Appl* 2025;264:125838. <http://dx.doi.org/10.1016/j.eswa.2024.125838>.
- [8] Nguyen VN, Tarelko W, Sharma P, El-Shafay AS, Chen W-H, Nguyen PQP, Nguyen XP, Hoang AT. Potential of explainable artificial intelligence in advancing renewable energy: challenges and prospects. *Energy Fuels* 2024;38(3):1692–712. <http://dx.doi.org/10.1021/acs.energyfuels.3c04343>.
- [9] Pes B. Ensemble feature selection for high-dimensional data: a stability analysis across multiple domains. *Neural Comput Appl* 2020;32(10):5951–73. <http://dx.doi.org/10.1007/s00521-019-04082-3>.
- [10] Dhal P, Azad C. A comprehensive survey on feature selection in the various fields of machine learning. *Appl Intell* 2022;52(4):4543–81. <http://dx.doi.org/10.1007/s10489-021-02550-9>.
- [11] Rodriguez-Galiano VF, Luque-Espinar JA, Chica-Olmo M, Mendes MP. Feature selection approaches for predictive modelling of groundwater nitrate pollution: An evaluation of filters, embedded and wrapper methods. *Sci Total Environ* 2018;624:661–72. <http://dx.doi.org/10.1016/j.scitotenv.2017.12.152>.
- [12] Guyon I, Elisseeff A. An introduction to variable and feature selection. *J Mach Learn Res* 2003;3(Mar):1157–82.
- [13] Cai J, Luo J, Wang S, Yang S. Feature selection in machine learning: A new perspective. *Neurocomputing* 2018;300:70–9. <http://dx.doi.org/10.1016/j.neucom.2017.11.077>.
- [14] Zebari R, Abdulazeez A, Zeebaree D, Zebari D, Saeed J. A comprehensive review of dimensionality reduction techniques for feature selection and feature extraction. *J Appl Sci Technol Trends* 2020;1(1):56–70. <http://dx.doi.org/10.38094/jastt1224>.
- [15] Chandrashekar G, Sahin F. A survey on feature selection methods. *Comput Electr Eng* 2014;40(1):16–28. <http://dx.doi.org/10.1016/j.compeleceng.2013.11.024>.
- [16] Subbiah SS, Chinnappan J. Deep learning based short term load forecasting with hybrid feature selection. *Electr Power Syst Res* 2022;210:108065. <http://dx.doi.org/10.1016/j.epsr.2022.108065>.
- [17] Xing Q, Huang X, Wang J, Wang S. A novel multivariate combined power load forecasting system based on feature selection and multi-objective intelligent optimization. *Expert Syst Appl* 2024;244:122970. <http://dx.doi.org/10.1016/j.eswa.2023.122970>.
- [18] Jiang B, Liu Y, Geng H, Wang Y, Zeng H, Ding J. A holistic feature selection method for enhanced short-term load forecasting of power system. *IEEE Trans Instrum Meas* 2022;72:1–11. <http://dx.doi.org/10.1109/TIM.2022.3219499>.
- [19] Aguida ME, Che Y, Mouhafid M, et al. Advanced filter-based feature selection for improved load identification in solar-powered aircraft distribution systems using XGBoost. *Measurement* 2025;246:116764. <http://dx.doi.org/10.1016/j.measurement.2025.116764>.
- [20] Saini P, Parida S. A novel probabilistic gradient boosting model with multi-approach feature selection and iterative seasonal trend decomposition for short-term load forecasting. *Energy* 2024;294:130975. <http://dx.doi.org/10.1016/j.energy.2024.130975>.
- [21] Chae YH, Kim SG, Choi J, Koo SR, Kim J. Enhancing nuclear power plant diagnostics: A comparative analysis of XAI-based feature selection methods for abnormal and emergency scenario detection. *Prog Nucl Energy* 2025;185:105759. <http://dx.doi.org/10.1016/j.pnucene.2025.105759>.
- [22] Dwivedi R, Dave D, Naik H, Singhal S, Omer R, Patel P, Qian B, Wen Z, Shah T, Morgan G, et al. Explainable AI (XAI): Core ideas, techniques, and solutions. *ACM Comput Surv* 2023;55(9):1–33. <http://dx.doi.org/10.1145/3561048>.
- [23] Nohara Y, Matsumoto K, Soejima H, Nakashima N. Explanation of machine learning models using shapley additive explanation and application for real data in hospital. *Comput Methods Programs Biomed* 2022;214:106584. <http://dx.doi.org/10.1016/j.cmpb.2021.106584>.
- [24] Van Zyl C, Ye X, Naidoo R. Harnessing explainable artificial intelligence for feature selection in time series energy forecasting: A comparative analysis of Grad-CAM and SHAP. *Appl Energy* 2024;353:122079.
- [25] Yang C, Guan X, Xu Q, Xing W, Chen X, Chen J, Jia P. How can SHAP (SHapley Additive exPlanations) interpretations improve deep learning based urban cellular automata model? *Comput Environ Urban Syst* 2024;111:102133. <http://dx.doi.org/10.1016/j.compenvurbysys.2024.102133>.
- [26] Jalalifar F, Delavar MR, Ghaderi SF, Alehashem SLM. SAGANConvLSTM: A novel spatio-temporal forecasting approach combining semivariogram-enhanced GAN and ConvLSTM for power load forecasting. *Comput Electr Eng* 2025;128:110718.
- [27] Zhou H, Zheng P, Dong J, Liu J, Nakanishi Y. Interpretable feature selection and deep learning for short-term probabilistic PV power forecasting in buildings using local monitoring data. *Appl Energy* 2024;376:124271.
- [28] Iqbal MS, Adnan M, Akbar MA, Bermak A. SALF: A self-adaptive learning framework for short-term load forecasting in smart grid. *Int J Energy Res* 2025;2025(1):6343205.
- [29] Iqbal MS, Adnan M, Mohamed SEG, Tariq M. A hybrid deep learning framework for short-term load forecasting with improved data cleansing and preprocessing techniques. *Results Eng* 2024;24:103560.
- [30] Choubey M, Chaurasiya RK, Yadav J. Robust electric load forecasting through ensemble learning: A stacking approach with empirical mode decomposition and transfer learning. *Comput Electr Eng* 2025;126:110511.
- [31] Fattah J, Ezzine L, Aman Z, El Moussami H, Lachhab A. Forecasting of demand using ARIMA model. *Int J Eng Bus Manag* 2018;10:1847979018808673. <http://dx.doi.org/10.1177/1847979018808673>.
- [32] Guo Y, Lai X, Gan M. Cyanobacterial biomass prediction in a shallow lake using the time series SARIMAX models. *Ecol Inform*. 2023;78:102292. <http://dx.doi.org/10.1016/j.ecoinf.2023.102292>.
- [33] Nurtas M, Zhantaev Z, Altaibek A. Earthquake time-series forecast in Kazakhstan territory: Forecasting accuracy with SARIMAX. *Procedia Comput Sci* 2024;231:353–8. <http://dx.doi.org/10.1016/j.procs.2023.12.216>.
- [34] Ledmaoui Y, El Maghraoui A, El Aroussi M, Saadane R, Chebak A, Chehri A. Forecasting solar energy production: A comparative study of machine learning algorithms. *Energy Rep* 2023;10:1004–12. <http://dx.doi.org/10.1016/j.egyri.2023.07.042>.
- [35] Benti NE, Chaka MD, Semie AG. Forecasting renewable energy generation with machine learning and deep learning: Current advances and future prospects. *Sustainability* 2023;15(9):7087. <http://dx.doi.org/10.3390/su15097087>.
- [36] Cahuantzi R, Chen X, Güttel S. A comparison of LSTM and GRU networks for learning symbolic sequences. In: Science and information conference. Springer; 2023. p. 771–85. http://dx.doi.org/10.1007/978-3-031-37963-5_53.
- [37] Lim B, Arık SÖ, Loeff N, Pfister T. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *Int J Forecast* 2021;37(4):1748–64. <http://dx.doi.org/10.1016/j.ijforecast.2021.03.012>.
- [38] Wu H, Xu J, Wang J, Long M. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Adv Neural Inf Process Syst* 2021;34:22419–30.
- [39] Zhou H, Zhang S, Peng J, Zhang S, Li J, Xiong H, Zhang W. Informer: Beyond efficient transformer for long sequence time-series forecasting. In: Proceedings of the AAAI conference on artificial intelligence, vol. 35, (12):2021, p. 11106–15.

- [40] Zohaib A, Akram F, Khalid S, Nawaz H, Rehman MU. Hybrid deep learning based load forecasting and AI-driven energy management for grid-connected multi-microgrids. *Comput Electr Eng* 2026;131:110915.
- [41] Kouki M, Osman N, Gafar M, El-Sehiemy RA. Multi-stage centralized energy management for interconnected microgrids: Hybrid forecasting, climate-resilient, and sustainable optimization. *Comput Model Eng Sci* 2025;145(3):3783.
- [42] Phyo P-P, Byun Y-C, Park N. Short-term energy forecasting using machine-learning-based ensemble voting regression. *Symmetry* 2022;14(1):160. <http://dx.doi.org/10.3390/sym14010160>.
- [43] Mumuni A, Mumuni F. Automated data processing and feature engineering for deep learning and big data applications: a survey. *J Inf Intell* 2025;3(2):113–53. <http://dx.doi.org/10.1016/j.jiixd.2024.01.002>.
- [44] Taylor SJ, Letham B. Forecasting at scale. *Amer Statist* 2018;72(1):37–45. <http://dx.doi.org/10.1080/00031305.2017.1380080>.
- [45] Du S, Li T, Yang Y, Horng S-J. Multivariate time series forecasting via attention-based encoder–decoder framework. *Neurocomputing* 2020;388:269–79. <http://dx.doi.org/10.1016/j.neucom.2019.12.118>.
- [46] Popov A. Feature engineering methods. In: *Advanced methods in biomedical signal processing and analysis*. Elsevier; 2023, p. 1–29. <http://dx.doi.org/10.1016/B978-0-323-85955-4.00004-1>.
- [47] Bommert A, Sun X, Bischl B, Rahnenführer J, Lang M. Benchmark for filter methods for feature selection in high-dimensional classification data. *Comput Statist Data Anal* 2020;143:106839. <http://dx.doi.org/10.1016/j.csda.2019.106839>.
- [48] Chemmakha M, Habibi O, Lazaar M. Improving machine learning models for malware detection using embedded feature selection method. *IFAC-PapersOnLine* 2022;55(12):771–6. <http://dx.doi.org/10.1016/j.ifacol.2022.07.406>.
- [49] Zheng H, Yuan J, Chen L. Short-term load forecasting using EMD-LSTM neural networks with a Xgboost algorithm for feature importance evaluation. *Energies* 2017;10(8):1168. <http://dx.doi.org/10.3390/en10081168>.
- [50] Chaturvedi S, Rajasekar E, Natarajan S, McCullen N. A comparative assessment of SARIMA, LSTM RNN and Fb Prophet models to forecast total and peak monthly energy demand for India. *Energy Policy* 2022;168:113097. <http://dx.doi.org/10.1016/j.enpol.2022.113097>.
- [51] Wang W, He N, Chen M, Jia P. Freight rate index forecasting with prophet model based on multi-dimensional significant events. *Expert Syst Appl* 2024;249:123451. <http://dx.doi.org/10.1016/j.eswa.2024.123451>.
- [52] Toharudin T, Pontoh RS, Caraka RE, Zahroh S, Lee Y, Chen RC. Employing long short-term memory and facebook prophet model in air temperature forecasting. *Comm Statist Simulation Comput* 2023;52(2):279–90. <http://dx.doi.org/10.1080/03610918.2020.1854302>.
- [53] Zhao D, Zhang R, Zhang H, He S. Prediction of global omicron pandemic using ARIMA, MLR, and Prophet models. *Sci Rep* 2022;12(1):18138. <http://dx.doi.org/10.1038/s41598-022-23154-4>.
- [54] Naima S, Su N, Kevin C, Victor G, Prem C. Forecasting container freight rates using the Prophet forecasting method. *Transp Policy* 2023;133:86–107. <http://dx.doi.org/10.1016/j.tranpol.2023.01.012>.
- [55] Rafferty G. Forecasting time series data with prophet: Build, improve, and optimize time series forecasting models using Meta's advanced forecasting tool. Packt Publishing Ltd; 2023.
- [56] Chen T, Guestrin C. Xgboost: A scalable tree boosting system. In: *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*. 2016, p. 785–94. <http://dx.doi.org/10.1145/2939672.2939785>.
- [57] Jafari S, Byun Y-C. Xgboost-based remaining useful life estimation model with extended kalman particle filter for lithium-ion batteries. *Sensors* 2022;22(23):9522. <http://dx.doi.org/10.3390/s22239522>.
- [58] Niazkar M, Menapace A, Brentan B, Piraei R, Jimenez D, Dhawan P, Righetti M. Applications of XGBoost in water resources engineering: A systematic literature review (Dec 2018–May 2023). *Environ Model Softw* 2024;174:105971. <http://dx.doi.org/10.1016/j.envsoft.2024.105971>.
- [59] Zhang P, Jia Y, Shang Y. Research and application of XGBoost in imbalanced data. *Int J Distrib Sens Networks* 2022;18(6):15501329221106935. <http://dx.doi.org/10.1177/15501329221106935>.
- [60] Uppalapati S, Paramasivam P, Kilari N, Chohan JS, Kanti PK, Vemanaboina H, Dabelo LH, Gupta R. Precision biochar yield forecasting employing random forest and XGBoost with Taylor diagram visualization. *Sci Rep* 2025;15(1):7105. <http://dx.doi.org/10.1038/s41598-025-91450-w>.
- [61] Li J, An X, Li Q, Wang C, Yu H, Zhou X, Geng Y-a. Application of XGBoost algorithm in the optimization of pollutant concentration. *Atmos Res* 2022;276:106238. <http://dx.doi.org/10.1016/j.atmosres.2022.106238>.
- [62] Jafari S, Yang J-H, Byun Y-C. Optimized XGBoost modeling for accurate battery capacity degradation prediction. *Results Eng* 2024;24:102786. <http://dx.doi.org/10.1016/j.rineng.2024.102786>.
- [63] Plevris V, Solorzano G, Bakas NP, Ben Seghier MEA. Investigation of performance metrics in regression analysis and machine learning-based prediction models. 2022. <http://dx.doi.org/10.23967/eccomas.2022.155>.
- [64] Zhou R, Zhang X. Short-term power load forecasting based on ARIMA-LSTM. In: *Journal of physics: conference series*, vol. 2803, (1):IOP Publishing; 2024, 012002.
- [65] Tantyoko H, Purnama SR, Vianita E. Multi-horizon short-term residential load forecasting using decomposition-based linear neural network. *Adv Sustain Sci Eng Technol* 2025;7(3):02503015.
- [66] Ehtearm M, Ghayoumi Zadeh H, Seifi A, Fayazi A, Dehghani M. Predicting hydropower production using deep learning CNN-ANN hybridized with gaussian process regression and salp algorithm. *Water Resour Manag* 2023;37(9):3671–97. <http://dx.doi.org/10.1007/s11269-023-03521-0>.
- [67] Gao R, Du L, Suganthan PN, Zhou Q, Yuen KF. Random vector functional link neural network based ensemble deep learning for short-term load forecasting. *Expert Syst Appl* 2022;206:117784.
- [68] Wan A, Chang Q, Khalil A-B, He J. Short-term power load forecasting for combined heat and power using CNN-LSTM enhanced by attention mechanism. *Energy* 2023;282:128274.
- [69] Wu H, Meng K, Fan D, Zhang Z, Liu Q. Multistep short-term wind speed forecasting using transformer. *Energy* 2022;261:125231.
- [70] Salinas D, Flunkert V, Gasthaus J, Januschowski T. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *Int J Forecast* 2020;36(3):1181–91.
- [71] Nazrul A. ProphetBoost: Electricity load forecasting. 2025. <https://github.com/SeasunAmin/ProphetBoost-A-Hybrid-Pipeline-for-Accurate-and-Transparent-Electricity-Load-Forecasting>. [Accessed 01 February 2025].